Rapport de projet ALGO

Achraf,Thomas,Elouen,Woody

Janvier 2024

Contents

1 Introduction **3**

2 Analyses **4**
2.1 Compression 4
2.2 Decompression 4

3 TAD Code Binaire **4**

4 TAD Octet **4**

5 TAD Statistiques **5**
5.1 TAD ArbreHuffman 5

6 FileDePriorité **6**

7 TAD TableDeCodage **7**

8 Conception préliminaire **9**
8.1 **Arbre Huffman** 9
8.2 **Code Binaire** 9
8.3 **Statistiques** 9
8.4 **Octet** 10

9 TAD Octet **10**
9.1 **FileDePriorite** 10
9.2 **TableDeCodage** 10
9.3 **Compression** 11
9.4 **Decompression** 11

10 Conception détaillée **12**
10.1 **Arbre Huffman** 12
10.2 Déclaration du type 12
10.3 arbrehuffman 12
10.4 fusionner 12
10.5 obtenirOccurences 12
10.6 fixerFilsDroit 13
10.7 fixerFilsGauche 13
10.8 estVide 13
10.9 estFeuille 13
10.10obtenirFilsGauche 13
10.11obtenirFilsDroit 13
10.12estPresent 14
10.13**Code Binaire** 14
10.14Déclaration du type 14
10.15creerCodeBinaire 14

10.16remplirCodeBinaire . . . 14
10.17libererCodeBinaire . . . 15
10.18estVide . . . 15
10.19obtenirIemeBit . . . 15
10.20concatener . . . 15
10.21codeBinaireToStr . . . 16
10.22obtenirCodeBinaire . . . 16
10.23obtenirCodeBinaireR . . . 16
10.24CbFromAH . . . 17
10.25codeBinaireToStr . . . 17
10.26**Statistiques** . . . 17
10.26.1 Déclaration du type . . . 17
10.26.2 statistique . . . 17
10.26.3 obtenirPonderation . . . 18
10.26.4 obtenirCaractere . . . 18
10.27**Octet** . . . 18
10.27.1 Déclaration du type . . . 18
10.27.2 creerOctet . . . 18
10.27.3 fixerBit . . . 18
10.27.4 iemeBit . . . 18
10.27.5 decaler . . . 19
10.27.6 bitsToStr . . . 19
10.27.7 strToBits . . . 19
10.27.8 encoder . . . 20
10.27.9 estVide . . . 20
10.28**FileDePriorite** . . . 20
10.29**TableDeCodage** . . . 20
10.29.1 Déclaration du type . . . 20
10.29.2 obtenirTableDeCodage . . . 20
10.29.3 obtenirCode . . . 20
10.29.4 obtenirCarac . . . 21
10.29.5 ajouter . . . 21
10.29.6 supprimer . . . 21
10.29.7 estPresent . . . 22
10.29.8 estVide . . . 22
10.30**Compression** . . . 22
10.31**Decompression** . . . 22

11 Code C et Tests Unitaires **23**
11.1 Code . . . 23
11.1.1 **Arbre Huffman** . . . 23
11.1.2 **Code Binaire** . . . 27
11.1.3 **Statistiques** . . . 31
11.1.4 **Octet** . . . 31
11.1.5 **FileDePriorite** . . . 36
11.1.6 **TableDeCodage** . . . 39

11.1.7 **Compression** 42
11.1.8 **Decompression** 45
11.2 Tests unitaires 47
11.2.1 **Arbre Huffman** 47
11.2.2 **Code Binaire** 51
11.2.3 **Statistiques** 55
11.2.4 **Octet** 62
11.2.5 **FileDePriorite** 64
11.2.6 **TableDeCodage** 65

12 Conclusion **70**
12.1 Répartition des tâches 70
12.1.1 Légende des Croix 70

1 Introduction

- Objectif 1 :

2 Analyses

2.1 Compression

2.2 Decompression

3 TAD Code Binaire

Nom:	Code Binaire	
Utilise:	TableDeCodage, Caractere, Bit	
Opérations:	obtenirCodeBinaire:	Arbre Huffman × **Caractere** → CodeBinaire
	estVide:	CodeBinaire ↛ **Booleen**
	obtenirIemeBit:	CodeBinaire × **NaturelNonNul** ↛ Bit
	longueur:	CodeBinaire → **Naturel**
Sémantiques:	obtenirCodeBinaire:	Permet d'obtenir un code binaire d'un caractère présent dans l'arbre
	estVide:	Cherche si un code binaire est vide ou non
	obtenirIemeBit:	Donne le ième bit d'un code binaire
	longueur:	Donne la longueur du code binaire
Préconditions:	obtenirCodeBinaire(a, car):	non(estVide(a))
	obtenirIemeBit(code, i):	n > 0 et n ≤ longueur(code)

4 TAD Octet

Nom:	Octet	
Utilise:	Chaine de caracteres, Code Binaire, Booleen, Bit	
Opérations:	creerOctet:	→ Octet
	modifier:	Octet **Naturel**Bit → Octet
	encoder:	Octet × Code Binaire**Naturel** ↛ Chaine de caracteres
	obtenirOctet:	Octet ↛ **Chaine de caracteres**
Sémantiques:	creerOctet:	Permet d'obtenir un octet avec tous ses bits à 0
	modifier:	Permet de modifier un bit dans un octet
	encoder:	Met un code binaire dans un octet à une certaine position
	obtenirOctet:	Donne la chaine de caractères correspondantes à un octet
Préconditions:	encoder(o, code,i):	non estVide(code), $0 \leq i \leq 7$

5 TAD Statistiques

Nom: Statistiques

Utilise: Caractère, Chaine de caracteres, Naturel

Opérations: statistiques: **Caractere** × **Naturel** → Statistiques

obtenirOccurences: Statistiques → **Naturel**

obtenirCaractere: Statistiques → **Caractere**

Sémantiques: statistiques: Créer une statistique à partir d'un caractère et d'un naturel

obtenirOccurences: Permet d'obtenir le nombre d'occurences d'une statistique

obtenirCaractere: Permet d'obtenir le caractère d'une statistique

5.1 TAD ArbreHuffman

Nom: Arbre Huffman

Utilise: statistiques,**Booleen**

Opérations:

arbreDeHuffman:	Statistiques → Arbre Huffman
fixerStatistique:	Arbre Huffman × Statistiques → Arbre Huffman
obtenirStat:	Arbre Huffman → Statistiques
obtenirCaractere:	Arbre Huffman → **Caractere**
fusionner:	Arbre Huffman × Arbre Huffman ↛ Arbre Huffman
obtenirPonderation:	Arbre Huffman → **Naturel**
fixerFilsDroit:	Arbre Huffman × Arebre Huffman ↛ Arbre Huffman
fixerFilsGauche:	Arbre Huffman × Arebre Huffman ↛ Arbre Huffman
obtenirFilsGauche:	Arbre Huffman ↛ Arbre Huffman
obtenirFilsDroit:	Arbre Huffman ↛ Arbre Huffman
estFeuille:	Arbre Huffman → **Booleen**
estVide:	Arbre Huffman → **Booleen**
estPresent:	Arbre Huffman → **Booleen**

Sémantiques: arbreDeHuffman: Crée un arbre de huffman à partir d'une statistique

	fixerStatistique:	Fixe une statistique à un arbre
	obtenirStat:	Retourne la valeur de la statistique d'un arbre
	obtenirCaractere:	Retourne le caractère associé à un arbre
	fusionner:	Fusionne deux arbres entre eux
	obtenirPonderation:	Permet d'obtenir la ponderation d'un arbre de huffman
	fixerFilsDroit:	Affecte la valeur au fils droit de l'arbre
	fixerFilsGauche:	Affecte la valeur au fils gauche de l'arbre
	obtenirFilsGauche:	Permet d'obtenir le fils gauche d'un arbre de huffman
	obtenirFilsDroit:	Permet d'obtenir le fils droit d'un arbre de huffman
	estFeuille:	Verifie si l'arbre de huffman est une feuille
	estVide:	Verifie si l'arbre est vide, si son champ caractère est vide
	estPresent:	Vérifie si un élément est présent dans un arbre
Préconditions:	fusionner(A):	estVide(A)
	fixerFilsDroit(r,ad):	non(estVide(r))
	fixerFilsGauche(r,ag):	non(estVide(r))
	obtenirFilsGauche(A):	non estUneFeuille(A)
	obtenirFilsDroit(A):	non estUneFeuille(A)

6 FileDePriorité

Nom: FileDePriorité

Utilise: Booleen

Opérations: fileDePriorite: $\rightarrow$ FileDePriorite

enfiler: FileDePriorite $\times$ Element $\rightarrow$ FileDePriorite

defiler: FileDePriorite $\rightarrow$ FileDePriorite

longueur: FiledePriorite $\rightarrow$ **Naturel**

nouvelleFile: FiledePriorite $\rightarrow$ FiledePriorite

estVide: FileDePriorite $\rightarrow$ **Booleen**

estPresent: FileDePriorite $\times$ Element $\rightarrow$ **Booleen**

obtenirElement: FileDePriorite $\rightarrow$ Element

Sémantiques: filedePriorite: Créer une file de priotité vide

enfiler:	Enfile un élément en fin de file
defiler:	Défile le dernier élément de la file
longueur:	Calcule la longueur de la file
nouvelleFile:	Créer la nouvelle file de priorité à partir de l'actuelle
estVide:	Cherche si la file de priotité est vide ou non
estPresent:	Cherche si un élément donné est présent dans la file ou non
obtenirElement:	Donne le dernier élément de la file

7 TAD TableDeCodage

Nom: TableDeCodage

Utilise: ArbreHuffmann

Opérations:

obtenirTableDeCodage:	ArbreHuffman $\rightarrow$ TableDeCodage
obtenirCode:	TableDeCodage $\times$ **Caractere** $\nrightarrow$ CodeBinaire
obtenirCarac:	TableDeCodage $\times$ CodeBinaire $\rightarrow$ **Caractere**
ajouter:	TableDeCodage $\times$ caractere $\times$ CodeBinaire $\rightarrow$ TableDeCodage
retirer:	TableDeCodage $\times$ **Caractere** $\rightarrow$ TableDeCodage
estPresent:	TableDeCodage $\times$ **Caractere** $\rightarrow$ **Booleen**
estVide:	TableDeCodage $\rightarrow$ **Booleen**

Sémantiques:

obtenirTableDeCodage:	Renvoie la table de codage grâce à un arbre de Huffman
obtenirCode:	Permet d'obtenir le code binaire d'un caractère grâce à la TableDeCodage
obtenirCarac:	Permet d'obtenir le caractere d'un code binaire grâce à la TableDeCodage
ajouter:	Permet d'ajouter une nouveeau caractère avec son code binaire dans la TableDeCodage
supprimerer:	Permet de supprimer un caractère de la TableDeCodage

	estPresent:	Permet de savoir si un caractère est présent dans la TableDeCodage
	estVide:	Permet de savoir si la TableDeCodage est vide

Préconditions: obtenirTableDeCodage(A): non estVide(A)
obtenirCode(t,c): estPresent(t,c)

8 Conception préliminaire

8.1 Arbre Huffman

fonction arbreDeHuffman (stat : Statistiques) **:** ˆArbreHuffman

procédure fixerStatistique (**E/S** arbre : ˆArbreHuffman **E** stat : Statistiques)

fonction obtenirStat (arbre : ArbreHuffman) **:** Statistiques

fonction obtenirPonderation (arbre : ArbreHuffman) **: Naturel**

fonction obtenirCaracter (arbre : ArbreHuffman) **: Caractere**

fonction fusionner (arbre1, arbre2 : ˆArbreHuffman) **:** ˆArbreHuffman
⌊**précondition(s)** non estVide(arbre1) ou non estVide(arbre2)

procédure fixerFilsGauche (**E/S** arbre : ˆArbreHuffman, **E** gauche : ˆArbreHuffman)

procédure fixerFilsDroit (**E/S** arbre : ˆArbreHuffman, **E** droit : ˆArbreHuffman)

fonction estFeuille (arbre : Arbre Huffman) **: Booleen**

fonction estVide (arbre: ˆArbreHuffman) **: Booleen**

fonction obtenirFilsGauche (arbre : ˆArbreHuffman) **:** ˆArbreHuffman
⌊**précondition(s)** non estFeuille(arbre)

fonction obtenirFilsDroit (arbre : ˆArbreHuffman) **:** ˆArbreHuffman
⌊**précondition(s)** non estFeuille(arbre)

fonction estPresent (arbre : ˆArbreHuffman, caractere : **Caractere) : Booleen**

8.2 Code Binaire

fonction obtenirCodeBinaire (a : Arbre Huffman, car : **Caractere) :** CodeBinaire
⌊**précondition(s)** non(estVide(a))

fonction estVide (code : CodeBinaire) **: Booleen**

fonction obtenirIemeBit (code : CodeBinaire, i : **NaturelNonNul) :** Bit
⌊**précondition(s)** i ¿ 0 et i ¡= longueur(code)

fonction longueur (code : CodeBinaire) **: Naturel**

8.3 Statistiques

fonction statistique (c : **Caractere**, n : **Naturel) :** Statistiques

fonction obtenirOccurences (stat : Statistiques) **: Naturel**

fonction obtenirCaractere (stat : Statistiques) **: Caractere**

8.4 Octet

9 TAD Octet

fonction obtenirStartPosition **() :**
fonction creerOctet **() :** Octet
procédure E/S octet : ˆOctet **E** pos : **Entier**, valeur : Bit **()**
fonction iemeBit **(**octet : Octet, i : **Entier) :** Bit
procédure decaler **(E/S** octet : ˆOctet**)**
fonction bitsToStr **(**octet : ˆOctet**) : Chaine de caracteres**
fonction strToBits **(**chaine : **Chaine de caracteres) :** Octet
procédure encoder **()**
fonction estVide **(Booleen) :** octet : Octet→

9.1 FileDePriorite

fonction filedePriorite **() :** FileDePriorite
procédure enfiler **(E/S** file : FileDePriorite , **E** arbre : ArbreHuffman**)**
fonction createNoeud **(**arbreOfLCA : ArbreHuffman**) :** Noeud
fonction trouverIndicePourInsere **(**arbre : ArbreHuffman, file : FileDePriorite**) : Naturel**
procédure defiler **(E/S** file : FileDePriorite**)**
⌊**précondition(s)** non(estVide(file))
fonction obtenirElement **(**file : FileDePriorite, i :**NaturelNonNul) :** ArbreHuffman
⌊**précondition(s)** i<file.longueur
procédure nouvelleFile **(E/S** file : FileDePriorite**)**
⌊**précondition(s)** non estVide(defiler(defiler(fdp)))

9.2 TableDeCodage

fonction obtenirTableDeCodage **(**a : ArbreHuffman**) :** TableDeCodage
⌊**précondition(s)** non estVide(a)
fonction obtenirCode **(**tab : TableDeCodage, c : **Caractere) :** CodeBinaire
⌊**précondition(s)** estPresent(tab,c)
fonction obtenirCarac **(**tab : TableDeCodage, code : CodeBinaire**) : Caractere**
procédure ajouter **(E/S** tab : TableDeCodage **E** c : **Caractere**, code : CodeBinaire**)**
procédure supprimer **(E/S** tab : TableDeCodage **E** c : **Caractere**, code : CodeBinaire**)**
fonction estPresent **(**tab : TableDeCodage, c : **Caractere) : Booleen**
fonction estVide **(**tab : TableDeCodage**) : Booleen**

9.3 Compression

fonction compresser (fic : FichierTexte) **:** FichierHuffman
fonction creerArbreHuffman (file : FileDePriorité) **:** Arbre Huffman
fonction creerTableDeCodage (arbre : Arbre Huffman) **:** TableDeCodage
fonction creerStatistiques (ensC : Ensemble de **Caractere**) **:** Ensemble de Statistiques
fonction creerFileDePriorite (fic : Fichier Texte) **:** FileDePriorite
fonction obtenirCaracteres (fic : FichierTexte) **:** Ensemble de **Caractere**
fonction obtenirOccurences (c : **Caractere**) **: Naturel**
procédure insererArbre (**E/S** file : FileDePriorite, **E** arbre : Arbre Huffman)
fonction creerArbre (stats : Statistiques) **:** Arbre Huffman

9.4 Decompression

fonction decompresser (fic : FichierHuffman) **:** FichierTexte
fonction creerArbreHuffman (stat : Statistiques) **:** Arbre Huffman
fonction obtenirFileDePriorite (fic : Fichier Huffman) **:** FileDePriorite
fonction obtenirStatistiques (fic : FichierHuffman) **:** Ensemble de Statistiques
procédure insererArbre (**E/S** file : FileDePriorite, **E** arbre : Arbre Huffman)
fonction creerArbre (stats : Statistiques) **:** Arbre Huffman

10 Conception détaillée

10.1 Arbre Huffman

10.2 Déclaration du type

Type ArbreHuffman = ^Racine
Type Racine = **Structure**
    filsGauche : ArbreHuffman
    filsDroit : ArbreHuffman
    statistique : Statistiques
finstructure

10.3 arbrehuffman

fonction arbrehuffman (stat : Statistiques) **:** ArbreHuffman
    Déclaration arbre : ArbreHuffman
debut
    arbre.filsGauche ← NIL
    arbre.filsDroit ← NIL
    arbre.statistique ← stat
    retourner arbre
fin

10.4 fusionner

fonction fusionner (arbre1 , arbre2 : ArbreHuffman) **:** ArbreHuffman
    ⌊**précondition(s)** non (estVide(arbre1) et estVide(arbre2))
    Déclaration arbre : ArbreHuffman
debut
    arbre ← arbrehuffman()
    arbre.statistique.caractere ← NULL
    arbre.statistique.ponderation ← obtenirOccurences(arbre1) + obtenirOccurences(arbre2)

    si obtenirOccurences(arbre1)<obtenirOccurences(arbre2) **alors**
        arbre.filsGauche ← arbre1
        arbre.filsDroit ← arbre2
    sinon
        arbre.filsDroit ← arbre1
        arbre.filsGauche ← arbre2
    finsi
    retourner arbre
fin

10.5 obtenirOccurences

fonction obtenirOccurences (arbre : ArbreHuffman) **: Naturel**

debut
  retourner arbre.statistique.ponderation
fin

10.6 fixerFilsDroit

procédure fixerFilsDroit **(E/S** arbre : ArbreHuffman **E** droit : ArbreHuffman**)**
debut
  arbre.filsDroit ← droit
fin

10.7 fixerFilsGauche

procédure fixerFilsGauche **(E/S** arbre : ArbreHuffman **E** gauche : ArbreHuffman**)**
debut
  arbre.filsGauche ← gauche
fin

10.8 estVide

fonction estVide **(**arbre : ArbreHuffman**) : Booleen**
debut
  retourner ((arbre.statistique.caractere = NULL) et (arbre.statistique.ponderation = 0))
fin

10.9 estFeuille

fonction estFeuille **(**arbre : ArbreHuffman**) : Booleen**
debut
  retourner (estVide(arbre.filsDroit) et estVide(arbre.filsGauche))
fin

10.10 obtenirFilsGauche

fonction obtenirFilsGauche **(**arbre : ArbreHuffman**) :** ArbreHuffman
  ⌊**précondition(s)** non estFeuille(arbre)
debut
  retourner arbre.filsGauche
fin

10.11 obtenirFilsDroit

fonction obtenirFilsDroit **(**arbre : ArbreHuffman**) :** ArbreHuffman

⌊**précondition(s)** non estFeuille(arbre)
debut
retourner arbre.filsDroit
fin

10.12 estPresent

fonction estPresent (arbre : ArbreHuffman, caractere : **Caractere) : Booleen**

debut
si arbre.statistiquescaractere = caractere **alors**
retourner Vrai
sinon
retourner estPresent(obtenirFilsDroit(arbre)) ou estPresent(obtenirFilsGauche(arbre))

finsi
fin

10.13 Code Binaire

10.14 Déclaration du type

Type CodeBinaire = **Structure**
bits : **Tableau**[1..MAX] **de** Bit
taille : **Naturel**
finstructure

10.15 creerCodeBinaire

fonction creerCodeBinaire (taille : **Naturel) :** CodeBinaire
Déclaration code : CodeBinaire, i : **Naturel**
debut
code.taille ← taille
pour i ←0 **à** 4 **faire**
code.bits[i] ← 0
finpour
retourner code
fin

10.16 remplirCodeBinaire

procédure remplirCodeBinaire **(E/S** code : CodeBinaire, **E** chaine : **Chaine de caracteres)**
⌊**précondition(s)** $longueur(chaine) \neq code.taille$
Déclaration i : **Naturel**
debut

```
    pour i ←0 à code.taille faire
        code.bits[i] ←
        obtenirIemeCaractere(chaine,i)
    finpour
fin
```

10.17 libererCodeBinaire

```
procédure libererCodeBinaire (E/S code : CodeBinaire)
debut
    desallouer(code.bits)
    code.taille ← 0
fin
```

10.18 estVide

```
fonction estVide (code : CodeBinaire) : Booleen
    Déclaration  taille : Naturel
debut
    taille ← code.taille
    retourner code = ""
fin
```

10.19 obtenirIemeBit

```
fonction obtenirIemeBit (code : CodeBinaire, i : NaturelNonNul) : Bit
    ⌊précondition(s)  i > 0 et i ≤ longueur(code)
debut
    retourner code[i]
fin
```

10.20 concatener

```
fonction concatener (code1, code2: CodeBinaire) : CodeBinaire
    Déclaration  res : CodeBinaire, i : Naturel
debut
    res.taille ← code1.taille + code2.taille
    pour i ←0 à code1.taille faire
        res.bits[i] ← code1.bits[i]
    finpour
    pour i ←0 à code2.taille faire
        res.bits[i + code1.taille] ← code2.bits[i]
    finpour
    retourner res
fin
```

10.21 codeBinaireToStr

fonction codeBinaireToStr (code: CodeBinaire) **: Chaine de caracteres**
  Déclaration str : **Chaine de caracteres**, i : **Naturel**
debut
  pour i ←0 **à** code.taille **faire**
    str ← str + code.bits[i]
  finpour
  retourner str
fin

10.22 obtenirCodeBinaire

fonction obtenirCodeBinaire (a : ArbreHuffman, car : **Caractere) : Chaine de caracteres**
  ⌊**précondition(s)** non ArbreHuffmanestVide(a).
debut
  si ArbreHuffmanestFeuille(a).**alors**
    retourner '0'
  sinon
    obtenirCodeBinaireR(a,car)
  finsi
fin

10.23 obtenirCodeBinaireR

fonction obtenirCodeBinaireR (a : ArbreHuffman, car : **Caractere) : Chaine de caracteres**
  ⌊**précondition(s)** non(estVide(a))
  Déclaration str,res : **Chaine de caracteres**, taille : **Naturel**
debut
  si estFeuille(a) **alors**
    retourner ""
  finsi
  si estPresent(obtenirFilsGauche(a),car) **alors**
    a ← obtenirFilsGauche(a)
    str ← obtenirCodeBinaireR(a,car)
    si str ≠ NULL **alors**
      res ← '0' + str
    finsi
    retourner res
  sinon
    a ← obtenirFilsDroit(a)
    str ← obtenirCodeBinaireR(a,car)
    si str ≠ NULL **alors**
      res ← '1' + str

```
        finsi
        retourner res
    finsi
fin
```

10.24 CbFromAH

```
fonction CbFromAH (a : ArbreHuffman, car : Caractere) : CodeBinaire
    Déclaration  res : CodeBinaire, str : Chaine de caracteres, taille :
                 Naturel
debut
    str ← obtenirCodeBinaire(a,car)
    taille ← longueur(str)
    res ← creerCodeBinaire(res,str)
    retourner res
fin
```

10.25 codeBinaireToStr

```
fonction codeBinaireToStr (code: CodeBinaire) : Chaine de caracteres
    Déclaration  str : Chaine de caracteres, i : Naturel
debut
    pour i ←0 à code.taille faire
        str ← str + code.bits[i]
    finpour
    retourner str
fin
```

10.26 Statistiques

10.26.1 Déclaration du type

```
Type Statistiques = Structure
    caractere : Caractere
    ponderation : Naturel
finstructure
```

10.26.2 statistique

```
fonction statistique (c : Caractere, n : Naturel) : Statistiques
    Déclaration  stat : Statistiques
debut
    stat.caracteres ← c
    stat.ponderation ← n
    retourner stat
fin
```

10.26.3 obtenirPonderation

fonction obtenirPonderation (stat : Statistiques) **: Naturel**
debut
  retourner stat.ponderation
fin

10.26.4 obtenirCaractere

fonction obtenirCaractere (stat : Statistiques) **: Caractere**
debut
  retourner stat.caractere
fin

10.27 Octet

10.27.1 Déclaration du type

Type Octet **= Structure**
  Bit : **Tableau**[1..8] **de** Bit
finstructure

10.27.2 creerOctet

fonction creerOctet **() :** Octet
  Déclaration res : Octet, i : **Naturel**
debut
  startPosition ← 0
  pour i ←1 **à** 8 **faire**
    res.bits[i] ← BITA0
  finpour
  retourner res
fin

10.27.3 fixerBit

procédure fixerBit **(E/S** octet : ˆOctet **E** pos : **Entier**, valeur : Bit**)**
  ⌊**précondition(s)** position*leq* 9
debut
  si position *leq* 9 **alors**
    octet→bits[position] ←
  finsi
fin

10.27.4 iemeBit

fonction iemeBit (octet : Octet, i : **Entier) :** Bit
debut

```
   retourner octet.bits[i]
fin
```

10.27.5 decaler

```
procédure decaler (E/S octet : ^Octet)
   Déclaration  i : Naturel
debut
   pour i ←1 à 8 faire
      octet→bits[i] ← bits[i+1]
   finpour
   octet→bits[8] ← BITA0
fin
```

10.27.6 bitsToStr

```
fonction bitsToStr (octet : ^Octet) : Chaine de caracteres
   Déclaration  str : ^Chaine de caracteres, i : Naturel
debut
   allouer(str)
   pour i ←1 à 8 faire
      si octet→bits[0] == BITA0 alors
         str[i] ← '0'
      sinon
         str[i] ← '1'
      finsi
   finpour
   str[8] ← '0'
fin
```

10.27.7 strToBits

```
fonction strToBits (chaine : Chaine de caracteres) : Octet
   Déclaration  octet : Octet, i : Naturel, longueurChaine : Entier
debut
   octet ← creerOctet()
   longueurChaine ← taille(chaine)
   pour i ←1 à longueurChaine-1 faire
      si chaine[i ]== '0'→alors
         octet.bits[i] ← BITA0
         incrementStartPosition()
      sinon
         octet.bits[i] ← BITA1
         incrementStartPosition()
      finsi
   finpour
fin
```

10.27.8 encoder

10.27.9 estVide

```
fonction estVide (Booleen) : octet : Octet→
   Déclaration  i : Naturel
debut
   pour i ←1 à 8 faire
      si octet→bits[i] neq BITA0 alors
         retourner Faux
      finsi
   finpour
   retourner Vrai
fin
```

10.28 FileDePriorite

10.28.1 Déclaration du type

```
Type Noeud = Structure
   arbre : ArbreHuffman
   suivant : Noeud
finstructure
Type FileDePriorite = Structure
   debut : Noeud
   longueur : Naturel
finstructure
```

10.29 fileDePriorite

```
fonction fileDePriorite () : FileDePriorite
   Déclaration  file:FileDePriorite
debut
   file.debut.← NIL
   file.longueur ← 0
   retourner file
fin
```

10.30 enfiler

```
procédure enfiler (E/S file : FileDePriorite, E arbre : ArbreHuffman)
   Déclaration  noeud,tmp : Noeud, positionToInsert,i : Naturel
debut
   noeud ← createNoeud(arbre)
   si estVide(file) alors
      file.debut ← noeud
      noeud.suivant ← NIL
```

```
        file.longueur ← file.longueur+1
    sinon
        positionToInsert ← trouverIndicePourInsere(arbre,file)
        si positionToInsert=0 alors
            noeud.suivant ← file.debut
            file.debut ← noeud
        sinon
            tmp ← file.debut
            pour i ←0 à positionToInsert−2 faire
                tmp ← tmp.suivant
            finpour
        finsi
        noeud.suivant ← tmp.suivant
        tmp.suivant ← noeud
        file.longueur ← file.longueur+1
    finsi
fin
```

10.31 createNoeud

```
fonction createNoeud (arbreOfLCA : ArbreHuffman) : Noeud
    Déclaration  noeud : Noeud
debut
    noeud.arbre ← arbreOfLCA
    noeud.suivant ← NIL
    retourner noeud
fin
```

10.32 trouverIndicePourInsere

```
fonction trouverIndicePourInsere (arbre : ArbreHuffman, file : FileDePriorite)
: Naturel
    Déclaration  tmp : Noeud, ponderationArbre, result : Naturel
debut
    tmp ← file.debut
    ponderationArbre ← obtenirPonderation(arbre)
    result ← 0
    tant que (tmp ≠ NIL) et (obtenirPonderation(tmp.arbre) > ponderation-
    Arbre) faire
        tmp ← tmp.suivant
        result ← result+1
    fintantque
    retourner result
fin
```

10.33 defiler

procédure defiler **(E/S** file : FileDePriorite**)**
⌊**précondition(s)** non(estVide(file))
Déclaration noeudActuel,noeudPrecedent : Noeud
debut
noeudActuel ← file.debut
noeudPrecedent ← NIL
tant que noeudActuel.suivant≠ NIL **faire**
noeudPrecedent ← noeudActuel
noeudActuel ← noeudActuel.suivant
fintantque
si noeudPrecedent ≠ NIL **alors**
noeudPrecedent.suivant ← NIL
sinon
file.debut ← NIL
finsi
desallouer(noeudActuel)
file.longueur ← file.longueur-1
fin

10.34 obtenirElement

fonction obtenirElement **(**file : FileDePriorite, i :**NaturelNonNul) :** Arbre-Huffman
⌊**précondition(s)** i<file.longueur
Déclaration temp : Noeud
debut
temp ← file.debut
pour j ←1 **à** i-1 **faire**
temp ← temp.suivant
finpour
retourner temp.arbre
fin

10.35 nouvelleFile

procédure nouvelleFile **(E/S** file : FileDePriorite**)**
Déclaration i : **Naturel**, arbre1,arbre2,arbreFusionne : ArbreHuffman
debut
si file.longueur > 2 **alors**
i ← file.longueur
arbre1 ← obtenirElement(file,i)
arbre2 ← obtenirElement(file,i-1)
defiler(file)
defiler(file)

```
        arbreFusionne ← fusionner(arbre1,arbre2)
        enfiler(file,arbreFusionne)
    finsi
fin
```

10.36 TableDeCodage

10.36.1 Déclaration du type

```
Type TableDeCodage = ^Noeud
Type Noeud = Structure
    code : CodeBinaire
    c : Caractere
    listeSuivante : TableDeCodage
finstructure
```

10.36.2 obtenirTableDeCodage

```
fonction obtenirTableDeCodage (a : Arbre Huffman) : TableDeCodage
    ⌊précondition(s)  non estVide(a)
    Déclaration  res : TableDeCodage
debut
    res.noeud.c ← obtenirCaractere(a.statistiques)
    res.noeud.code ← obtenirCodeBinaire(a,res.noeud.c)
    res.noeud.listeSuivante ← NIL
    retourner res
fin
```

10.36.3 obtenirCode

```
fonction obtenirCode (tab: TableDeCodage, c : Caractere) : CodeBinaire
    ⌊précondition(s)  estPresent(tab,c)
debut
    si non(estVide(tab)) alors
        si tab.noeud.c=c alors
            retourner tab.noeud.code
        sinon
            retourner obtenirCode(tab.noeud.listeSuivante,c)
        finsi
    finsi
fin
```

10.36.4 obtenirCarac

```
fonction obtenirCarac (tab: TableDeCodage, code : CodeBinaire) : Carac-
tere
debut
```

```
    si non(estVide(tab)) alors
        si tab.noeud.code=code  alors
            retourner tab.noeud.c
        sinon
            retourner obtenirCode(tab.noeud.listeSuivante,code)
        finsi
    finsi
fin
```

10.36.5 ajouter

```
procédure ajouter (E/S tab : TableDeCodage, E c : Caractere, code : CodeBinaire)
    Déclaration  temp : TableDeCodage
debut
    temp ← tab
    tab.noeud.c ← c
    tab.noeud.code ← code
    tab.noeud.listeSuivante ← temp.noeud
fin
```

10.36.6 supprimer

```
procédure supprimer (E/S tab : TableDeCodage, E c : Caractere, code : CodeBinaire)
    ⌊précondition(s)  non(estVide(tab))
    Déclaration  temp,temp2 : TableDeCodage
debut
    si non(estVide(tab.noeud.listeSuivante)) alors
        si temp.noeud.c = c alors
            temp.noeud ← temp.noeud.listeSuivante
            tab.noeud.listeSuivante ← temp.noeud
        sinon
            temp2.noeud ← temp.noeud.listeSuivante  supprimer(temp2.listeSuivante,c,code)
            tab.noeud.listeSuivante ← temp.noeud
        finsi
    finsi
fin
```

10.36.7 estPresent

```
fonction estPresent (tab : TableDeCodage, c : Caractere) : Booleen
debut
    si estVide(tab) alors
        retourner FAUX
    finsi
```

```
    si tab.noeud.c = c alors
        retourner VRAI
    sinon
        temp.noeud ← tab.noeud.listeSuivante
        retourner estPresent(temp,c)
    finsi
fin
```

10.36.8 estVide

```
fonction estVide (tab : TableDeCodage) : Booleen
debut
    retourner tab.noeud=NIL
fin
```

10.37 Compression

10.38 Decompression

11 Code C et Tests Unitaires

11.1 Code

11.1.1 Arbre Huffman

Listing 1: ArbreHuffman.c

```c
#include<stdio.h>
#include<stdbool.h>
#include <stddef.h>
#include <stdlib.h>
#include "Statistiques.h"
#include "ArbreHuffman.h"



//v
ArbreHuffman* AH_arbreDeHuffman(Statistiques stat) {
    ArbreHuffman *arbre = malloc(sizeof(ArbreHuffman))
        ;
    if (arbre != NULL) {
        arbre->statistiques = stat;
        arbre->filsGauche = NULL;
        arbre->filsDroit = NULL;
    }
    return arbre;
}
//v
void AH_fixerStatistique(ArbreHuffman *arbre,
   Statistiques stat){
    arbre->statistiques=stat;
}
//v
Statistiques AH_obtenirStat(ArbreHuffman arbre){
    return arbre.statistiques;
}

//v
int AH_obtenirPonderation(ArbreHuffman arbre){
    return S_obtenirPonderation(AH_obtenirStat(arbre))
        ;
}



//v
```

```
37 char AH_obtenirCaracter(ArbreHuffman arbre){
38     return arbre.statistiques.caracter;
39 }
40
41 ArbreHuffman* AH_fusionner(ArbreHuffman* arbre1,
       ArbreHuffman* arbre2) {
42     Statistiques stat ;
43     stat.caracter = ' ';
44     stat.ponderation = S_obtenirPonderation(arbre1->
           statistiques)+S_obtenirPonderation(arbre2->
           statistiques);
45     ArbreHuffman* res = AH_arbreDeHuffman(stat);
46     if (AH_obtenirPonderation(*arbre1)<
           AH_obtenirPonderation(*arbre2)){
47     AH_fixerFilsGauche(res,arbre1);
48     AH_fixerFilsDroit(res,arbre2);
49     }
50     else {
51         AH_fixerFilsGauche(res,arbre2);
52         AH_fixerFilsDroit(res,arbre1);
53     }
54     return res;
55 }
56
57
58 void AH_fixerFilsGauche(ArbreHuffman *arbre,
       ArbreHuffman *filsGauche) {
59     if (arbre != NULL) {
60         arbre->filsGauche = filsGauche;
61     }
62 }
63
64 void AH_fixerFilsDroit(ArbreHuffman *arbre,
       ArbreHuffman *filsDroit) {
65     if (arbre != NULL) {
66         arbre->filsDroit = filsDroit;
67     }
68 }
69
70 ArbreHuffman* AH_obtenirFilsGauche(ArbreHuffman *arbre
       ) {
71     return (arbre != NULL) ? arbre->filsGauche : NULL;
72 }
73
74 ArbreHuffman* AH_obtenirFilsDroit(ArbreHuffman *arbre)
       {
```

```
75      return (arbre != NULL) ? arbre->filsDroit : NULL;
76  }
77
78  bool AH_estVide(ArbreHuffman *arbre) {
79      return arbre==NULL;
80  }
81
82  bool AH_estFeuille(ArbreHuffman arbre) {
83      return (AH_estVide(AH_obtenirFilsGauche(&arbre))
            && AH_estVide(AH_obtenirFilsDroit(&arbre)));
84  }
85
86  bool AH_estPresent(ArbreHuffman *arbre, char caractere
        ) {
87      if (!AH_estVide(arbre)){
88          if (arbre->statistiques.caracter == caractere)
                {
89              return true;
90          }
91          else {
92              return (AH_estPresent(AH_obtenirFilsDroit(
                    arbre),caractere) || AH_estPresent(
                    AH_obtenirFilsGauche(arbre),caractere))
                    ;
93          }
94      }
95      return false;
96  }
97
98  void AH_afficherArbre(ArbreHuffman *arbre) {
99      if (AH_estVide(arbre)) {
100         printf("L'arbre est vide.\n");
101         return;
102     }
103
104     // Utilisation d'une pile pour l'itération
105     typedef struct {
106         ArbreHuffman *noeud;
107         int niveau;
108     } StackNode;
109
110     // Initialisation de la pile
111     StackNode *pile = NULL;
112     int capacitePile = 10;
113     int taillePile = 0;
114
```

```
115     // Empilement du noeud racine
116     StackNode racineNode = {arbre, 0};
117     pile = realloc(pile, capacitePile * sizeof(
            StackNode));
118     pile[taillePile++] = racineNode;
119
120     while (taillePile > 0) {
121         // Dépilement du noeud
122         StackNode noeudCourant = pile[--taillePile];
123
124         // Affichage du noeud
125         for (int i = 0; i < noeudCourant.niveau; ++i)
                {
126             printf("    ");
127         }
128         printf("%c/%d\n", AH_obtenirCaracter(*
                noeudCourant.noeud), AH_obtenirPonderation
                (*noeudCourant.noeud));
129
130         // Empilement du fils droit si non nul
131         ArbreHuffman *fd = AH_obtenirFilsDroit(
                noeudCourant.noeud);
132         if (fd != NULL) {
133             if (taillePile >= capacitePile) {
134                 capacitePile *= 2;
135                 pile = realloc(pile, capacitePile *
                        sizeof(StackNode));
136             }
137             pile[taillePile++] = (StackNode){fd,
                    noeudCourant.niveau + 1};
138         }
139
140         // Empilement du fils gauche si non nul
141         ArbreHuffman *fg = AH_obtenirFilsGauche(
                noeudCourant.noeud);
142         if (fg != NULL) {
143             if (taillePile >= capacitePile) {
144                 capacitePile *= 2;
145                 pile = realloc(pile, capacitePile *
                        sizeof(StackNode));
146             }
147             pile[taillePile++] = (StackNode){fg,
                    noeudCourant.niveau + 1};
148         }
149     }
150
```

```
151     // Libération de la mémoire de la pile
152     free(pile);
153 }
154 ArbreHuffman* copierArbreHuffman(ArbreHuffman* arbre)
        {
155     if (arbre == NULL) {
156         return NULL;
157     }
158 
159     // Copie récursive du noeud actuel
160     Statistiques stat= S_statistique(
            AH_obtenirCaracter(*arbre),
            AH_obtenirPonderation(*arbre));
161     ArbreHuffman* nouveauNoeud = AH_arbreDeHuffman(
            stat);
162 
163     // Appels récursifs pour copier les sous-arbres
        gauche et droit
164     nouveauNoeud->filsGauche = copierArbreHuffman(
            AH_obtenirFilsGauche(arbre));
165     nouveauNoeud->filsDroit = copierArbreHuffman(
            AH_obtenirFilsDroit(arbre));
166 
167     return nouveauNoeud;
168 }
```

11.1.2 Code Binaire

Listing 2: CodeBinaire.c

```
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include "CodeBinaire.h"
4  #include <stdlib.h>
5  #include <string.h>
6  #define BITA_END '\0'
7 
8  bool CB_estVide(CodeBinaire cb) {
9      int  taille = cb.taille;
10     return taille == 0;
11 }
12 
13 CodeBinaire CB_creerCodeBinaire(int taille) {
14     CodeBinaire code;
15     code.bits = malloc(taille * sizeof(Bit));
```

```
16      code.taille = taille;
17      for (size_t i = 0; i < 4; ++i) {
18          code.bits[i] = BITA0;
19      }
20      return code;
21  }
22
23  void CB_remplirCodeBinaire(CodeBinaire* code, const
        char* chaine) {
24      int longueurChaine = strlen(chaine);
25
26
27      if (longueurChaine != code->taille) {
28          fprintf(stderr, "Erreur : La longueur de la
                chaîne ne correspond pas á la taille du
                CodeBinaire.\n");
29          exit(EXIT_FAILURE);
30      }
31
32      for (size_t i = 0; i < code->taille; ++i) {
33          code->bits[i] = (chaine[i] == '0') ? BITA0 :
                BITA1;
34      }
35  }
36
37
38  void CB_libererCodeBinaire(CodeBinaire* code) {
39      free(code->bits);
40      code->taille = 0;
41  }
42
43  Bit CB_obtenirIemeBit(CodeBinaire code,int i) {
44      if (i < code.taille) {
45          return code.bits[i];
46      } else {
47          fprintf(stderr, "Erreur : Indice hors limites
                .\n");
48          exit(EXIT_FAILURE);
49      }
50  }
51
52
53  CodeBinaire CB_concatener(const CodeBinaire* code1,
        const CodeBinaire* code2) {
54      CodeBinaire resultat;
55      resultat.taille = code1->taille + code2->taille;
```

```
56      resultat.bits = malloc(resultat.taille * sizeof(
            Bit));
57
58      for (size_t i = 0; i < code1->taille; ++i) {
59          resultat.bits[i] = code1->bits[i];
60      }
61
62      for (size_t i = 0; i < code2->taille; ++i) {
63          resultat.bits[code1->taille + i] = code2->bits
                [i];
64      }
65
66      return resultat;
67  }
68
69  char* CB_codeBinaireToStr(const CodeBinaire* code) {
70      char* str = malloc((code->taille + 1) * sizeof(
            char));
71      for (int i = 0; i < code->taille; ++i) {
72          str[i] = (code->bits[i] == BITA0) ? '0' : '1';
73      }
74      str[code->taille] = '\0';
75      return str;
76  }
77
78
79  CodeBinaire CB_CbFromAH(ArbreHuffman *a, char car){
80      char* result = AH_obtenirCodeBinaire(a,car);
81      int taille = strlen(result);
82      CodeBinaire CB_result = CB_creerCodeBinaire(taille
            );
83      CB_remplirCodeBinaire(&CB_result,result);
84      return CB_result;
85  }
86
87  char* AH_obtenirCodeBinaire(ArbreHuffman *a, char car)
        {
88      if (!AH_estVide(a)) {
89          if (AH_estFeuille(*a)) {
90              return strdup("0");
91          } else {
92              return obtenirCodeBinaireR(a, car);
93          }
94      }
95
96      return NULL;
```

```
97  }
98
99  char* obtenirCodeBinaireR(ArbreHuffman *a, char car) {
100     if (!AH_estVide(a)) {
101         if (AH_estFeuille(*a)) {
102             return strdup("");  // Retourne une chaîne
                    vide allouée dynamiquement
103         }
104         if (AH_estPresent(AH_obtenirFilsGauche(a), car
                )) {
105             a = AH_obtenirFilsGauche(a);
106             char* resultatGauche = obtenirCodeBinaireR
                    (a, car);
107             if (resultatGauche != NULL) {
108                 // Alloue suffisamment de mémoire pour
                        "0" suivi de la chaîne de résultat
109                 char* resultat = malloc(strlen("0") +
                        strlen(resultatGauche) + 1);
110                 if (resultat != NULL) {
111                     strcpy(resultat, "0");
112                     strcat(resultat, resultatGauche);
113                 }
114                 free(resultatGauche);  // Libère la mé
                        moire allouée dynamiquement pour la
                        chaîne de résultat gauche
115                 return resultat;
116             }
117         } else {
118             a = AH_obtenirFilsDroit(a);
119             char* resultatGauche = obtenirCodeBinaireR
                    (a, car);
120             if (resultatGauche != NULL) {
121                 // Alloue suffisamment de mémoire pour
                        "1" suivi de la chaîne de résultat
122                 char* resultat = malloc(strlen("1") +
                        strlen(resultatGauche) + 1);
123                 if (resultat != NULL) {
124                     strcpy(resultat, "1");
125                     strcat(resultat, resultatGauche);
126                 }
127                 free(resultatGauche);  // Libère la mé
                        moire allouée dynamiquement pour la
                        chaîne de résultat gauche
128                 return resultat;
129             }
130         }
```

```
131     }
132     return NULL;
133 }
```

11.1.3 Statistiques

Listing 3: Statistiques.c

```
1  #include<stdio.h>
2  #include "Statistiques.h"
3
4  Statistiques S_statistique(char c, int n) {
5      Statistiques res;
6      res.caracter = c;
7      res.ponderation = n ;
8      return res;
9  }
10
11 int S_obtenirPonderation(Statistiques stat) {
12     return stat.ponderation;
13 }
14
15 char S_obtenirCaractere(Statistiques stat) {
16     return stat.caracter;
17 }
```

11.1.4 Octet

Listing 4: Octet.c

```
1  #include<stdio.h>
2  #include<stdbool.h>
3  #include <string.h>
4  #include "Octet.h"
5  #include "CodeBinaire.h"
6  #include <stdlib.h>
7  #include "ArbreHuffman.h"
8  static int startPosition=0;
9  static bool currentisFull;
10 static Octet bitRestant;
11 static int mostImportantBitRestant=8;
12
13 Octet O_obtenirBitRestants(){
14     //printf("bit restant : %s\n", O_octetToStr(&
       bitRestant));
```

```
15      return bitRestant;
16  }
17
18  int O_obtenirMostImportantBitRestant(){
19      return mostImportantBitRestant;
20  }
21
22  void O_incrementStartPosition(){
23      startPosition++;
24  }
25
26  int O_obtenirStartPosition(){
27      return startPosition;
28  }
29
30  Octet O_creerOctet(){
31      Octet result;
32      startPosition=0;
33      currentisFull = false;
34      for (size_t i = 0; i < 8; ++i) {
35          result.bits[i] = BITA0;
36      }
37      return result;
38  }
39
40  void O_fixerBit(Octet* octet, int position, Bit valeur
        ) {
41      if (position < 8) {
42          octet->bits[position] = valeur;
43      } else {
44          fprintf(stderr, "Erreur : Position hors
                limites.\n");
45      }
46      //return *octet;
47  }
48
49  bool O_currentisFull(){
50      return currentisFull;
51  }
52
53
54  Bit O_iemeBit(Octet octet, int i){
55      return octet.bits[i];
56  }
57
58
```

```
59 //decale a gauche pou libere un bit pour insertion
       dans bitRestant
60 void O_decaler(Octet *octet){
61     for(int i=0 ;i<7;i++){
62         octet->bits[i]=octet->bits[i+1];
63     }
64     octet->bits[7]= BITA0;
65 }
66
67
68 char* O_bitsToStr(const Octet* octet) {
69     char* str = malloc(9 * sizeof(char));  // +1 pour
           le caractère nul
70     for (int i = 0; i < 8; ++i) {
71         str[i] = (octet->bits[i] == BITA0) ? '0' : '1'
               ;
72     }
73     str[8] = '\0';  // Terminer la chaîne avec le
           caractère nul
74     return str;
75 }
76
77
78 Octet O_strToBits(const char* chaine){
79     Octet octet = O_creerOctet();
80
81     int longueurChaine = strlen(chaine);
82     for (int i = 0; i < longueurChaine; ++i) {
83         octet.bits[i] = (chaine[i] == '0') ? BITA0 :
               BITA1;
84         O_incrementStartPosition();
85     }
86
87
88     return octet;
89
90 }
91
92 //01010100
93 //00000001
94 //00000000
95 Octet O_encoder(Octet octet,CodeBinaire cb){
96     Bit bit;
97     //printf("je ne suis pas encore dans une condition
           \n");
98     //cas ou y a des bits dans l'octet des bit restant
```

```
99      // On insere autant de bit possible dans l'octet;
100     if (mostImportantBitRestant<=7 && !currentisFull)
            {
101         for (int i=mostImportantBitRestant ;i<8;i++){
102             bit = O_iemeBit(bitRestant,i);
103             //octet=O_fixerBit(&octet,startPosition,
                    bit);
104             O_fixerBit(&octet,startPosition,bit);
105             startPosition++;
106             //bitRestant = O_fixerBit(&bitRestant,
                    mostImportantBitRestant,BITA0);
107             O_fixerBit(&bitRestant,
                    mostImportantBitRestant,BITA0);
108             mostImportantBitRestant++;
109         }
110     }
111
112     //printf("je suis dans la deuxieme condition\n");
113     //Apres On passe au bits du code binaire ,on les
            insere s'il reste de la place dans octet,
114     //sinon on les insere dans l'octet bitRestant
115     for (int i=0;i< cb.taille;i++){
116         Bit bit = CB_obtenirIemeBit(cb,i);
117         //printf("start position after strtobits is :
                %d\n",O_obtenirStartPosition());
118         if(!currentisFull /*ca veut dire il reste de
                la place dans l'octet principale*/){
119             //printf("Je ne dois pas inserer dans l'
                    Octet bitRestants\n");
120             //octet = O_fixerBit(&octet,startPosition,
                    bit);
121
122             O_fixerBit(&octet,startPosition,bit);
123
124             startPosition++;
125         }
126         else{/*Cad il ne reste pas de la place */
127             //printf("Je dois inserer dans l'octet
                    bitRestant\n");
128             //bitRestant = O_decaler(bitRestant);
129             O_decaler(&bitRestant);
130             //printf("Octet avant insertion decaler :
                    %s\n", O_octetToStr(&bitRestant));
131             //bitRestant = O_fixerBit(&bitRestant,7,
                    bit);
132             O_fixerBit(&bitRestant,7,bit);
```

```
133             //printf("Octet decaler : %s\n",
                    O_octetToStr(&bitRestant));
134             mostImportantBitRestant--;
135         }
136         if(startPosition >7) {
137             currentisFull =true;
138             //quand y a plus de place dans l'octet on
                    prend un autre donc start position est
                    remis a 0;
139             // startPosition =0;
140         }
141     }
142
143
144
145
146     //Si apres avoir inserer les bit restants des
            appels precedants et les bits de code binaire
147     // et qu'il reste plus de place dans octet on
            remet start position a 0 pour le prochain octet
148     //qu'on va traite
149
150     if(startPosition >7) {
151         currentisFull =true;
152         //quand y a plus de place dans l'octet on
                prend un autre donc start position est
                remis a 0;
153         // startPosition =0;
154     }
155     return octet;
156 }
157
158 bool O_estVide(const Octet* octet) {
159     for (size_t i = 0; i < 8; ++i) {
160         if (octet->bits[i] != BITA0) {
161             return false;
162         }
163     }
164     return true;
165 }
166
167
168 Octet O_inverser(Octet o){
169     Octet result = O_creerOctet();
170     for(int i=0;i<8;i++){
171         O_fixerBit(&result,i,O_iemeBit(o,7-i));
```

```
172     }
173     return result;
174 }
```

11.1.5 FileDePriorite

Listing 5: FileDePriorite.c

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<stdbool.h>
4  #include "FileDePriorite.h"
5  #include "ArbreHuffman.h"
6
7  bool FDP_estVide(FileDePriorite *fp) {
8      return fp->longueur==0;
9  }
10
11 FileDePriorite FDP_fileDePriorite() {
12     FileDePriorite file;
13     file.debut = NULL;
14         file.longueur = 0;
15     return file;
16 }
17
18 void FDP_enfiler(FileDePriorite *file,ArbreHuffman*
       arbre) {
19     Noeud* noeud = FDP_createNoeud(arbre);
20     if (FDP_estVide(file)) {
21         file->debut=noeud;
22         noeud->suivant=NULL;
23         file->longueur++;
24     }
25     else {
26         int positionToInsert = trouverIndicePourInsere
               (arbre, file);
27         if (positionToInsert == 0) {
28             noeud->suivant = file->debut;
29             file->debut = noeud;
30         } else {
31             Noeud *tmp = file->debut;
32             for (int i = 0; i < positionToInsert - 1;
                   i++) {
33                 tmp = tmp->suivant;
34             }
```

```
35              noeud->suivant = tmp->suivant;
36              tmp->suivant = noeud;
37          }
38          file->longueur++;
39      }
40  }
41
42  Noeud* FDP_createNoeud(ArbreHuffman* arbreOfLCA){
43      Noeud* noeud = malloc(sizeof(Noeud));
44      noeud->arbre=*arbreOfLCA;
45      noeud->suivant=NULL;
46      return noeud;
47  }
48
49  void FDP_afficherFileDEPriorite(FileDePriorite* file)
        {
50      Noeud *actuel = file->debut;
51      printf("la liste est: \n");
52
53      while (actuel != NULL) {
54          AH_afficherArbre(&actuel->arbre);
55          printf("->");
56          actuel = actuel->suivant;
57      }
58      if(actuel == NULL) printf("NULL");
59  }
60
61  //void fixerNoeud(FileDePriorite* file,Noeud* noeud,
        int position){
62  //      Noeud* tmp= file->debut;
63  //      for(int i=0;i<position-1;i++){
64  //          tmp= tmp->suivant;
65  //      }
66  //      Noeud* tmp1= tmp->suivant;
67  //      tmp->suivant=noeud;
68  //      noeud->suivant=tmp1;
69  //}
70
71  int trouverIndicePourInsere(ArbreHuffman* arbre,
        FileDePriorite* file){
72      Noeud *tmp = file->debut;
73      int ponderationArbre = AH_obtenirPonderation(*
            arbre);
74      int result = 0;
75
```

```
76      while (tmp != NULL && AH_obtenirPonderation(tmp->
            arbre) > ponderationArbre) {
77          tmp = tmp->suivant;
78          result++;
79      }
80
81      return result;
82  }
83
84  void FDP_defiler(FileDePriorite *file){
85          if (!FDP_estVide(file)) {
86          Noeud *noeudActuel = file->debut;
87          Noeud *noeudPrecedent = NULL;
88
89          // Parcourir la file jusqu'au dernier élément
90          while (noeudActuel->suivant != NULL) {
91              noeudPrecedent = noeudActuel;
92              noeudActuel = noeudActuel->suivant;
93          }
94
95          // Si la file contient plus d'un élément
96          if (noeudPrecedent != NULL) {
97              noeudPrecedent->suivant = NULL;
98          } else {
99              // Si la file ne contient qu'un seul élé
                    ment
100             file->debut = NULL;
101         }
102
103         free(noeudActuel);
104         file->longueur--;
105     }
106 }
107
108 // void FDP_defilerR(FileDePriorite *file) {
109 //     ListeChaineeArbre *temp = file->debut;
110 //     if (FDP_estVide_LCA(temp->listeSuivante)){
111 //         free(temp);
112 //         file->debut = NULL;
113 //     }
114 //     else{
115 //         file->debut = temp->listeSuivante;
116 //         free(temp);
117 //         FDP_defilerR(file);
118 //     }
119 // }
```

```
120 // void FDP_defiler(FileDePriorite *file){
121 //     FDP_defilerR(file);
122 //     file->longueur = file->longueur-1;
123 // }
124
125 ArbreHuffman FDP_obtenirElement(FileDePriorite file,
        int i) {
126     Noeud* temp;
127     temp = file.debut;
128     for (int j=1;j<i;j++) {
129          temp = temp->suivant;
130      }
131     return temp->arbre;
132 }
133
134 void FDP_nouvelleFile(FileDePriorite* file) {
135    if (file->longueur < 2) {
136         return;
137     }
138     int i = file->longueur;
139     ArbreHuffman arbre1 = FDP_obtenirElement(*file, i)
           ;
140     ArbreHuffman arbre2 = FDP_obtenirElement(*file, i
           - 1);
141
142     FDP_defiler(file);
143     FDP_defiler(file);
144
145     ArbreHuffman arbreFusionne = *AH_fusionner(&arbre1
           ,&arbre2);
146     FDP_enfiler(file, &arbreFusionne);
147 }
```

11.1.6 TableDeCodage

Listing 6: TableDeCodage.c

```
1 #include<stdio.h>
2 #include "TableDeCodage.h"
3 #include<stdbool.h>
4 #include <stdlib.h>
5
6 TableDeCodage* TDC_obtenirTableDeCodage() {
7     TableDeCodage* table = malloc(sizeof(TableDeCodage
          ));
```

```
8       if (table != NULL) {
9           table->ligne = NULL;
10      }
11      return table;
12  }
13
14  CodeBinaire* TDC_obtenirCode(TableDeCodage tab,char c)
        {
15      Ligne* current = tab.ligne;
16      while (current != NULL) {
17          if (current->c == c) {
18              return &current->code;
19          }
20          current = current->listeSuivante;
21      }
22      return NULL;  // Le caractère n'a pas été trouvé
            dans la table.
23  }
24
25  char TDC_obtenirCarac(TableDeCodage tab,CodeBinaire
        code) {
26      if (!(TDC_estVide(tab))) {
27          //Il faut ajouter une methode equlas dans
                codeBinaire
28          if (CB_codeBinaireToStr(&tab.ligne->code) ==
                CB_codeBinaireToStr(&code)) {
29              return tab.ligne->c;
30          }
31          else {
32              return TDC_obtenirCarac((TableDeCodage){
                    tab.ligne->listeSuivante},code);
33          }
34
35      }
36      return 'c';
37  }
38
39  void TDC_ajouter(TableDeCodage *tab,char c,CodeBinaire
        *code) {
40      Ligne* nouvelleLigne = malloc(sizeof(Ligne));
41      if (nouvelleLigne != NULL) {
42          nouvelleLigne->code = *code;
43          nouvelleLigne->c = c;
44          nouvelleLigne->listeSuivante = tab->ligne;
45          tab->ligne = nouvelleLigne;
46      }
```

```
47 }
48
49 void TDC_supprimer(TableDeCodage *tab, char *c,
       CodeBinaire *code) {
50     TableDeCodage temp, temp2;
51     temp2.ligne = tab->ligne->listeSuivante;
52     if (!(TDC_estVide(temp2))) {
53         if (&temp.ligne->c == c) {
54             temp.ligne = temp.ligne->listeSuivante;
55             tab->ligne->listeSuivante = temp.ligne;
56         }
57         else {
58             temp2.ligne = temp.ligne->listeSuivante;
59             TDC_supprimer(&temp2, c, code);
60             tab->ligne->listeSuivante = temp.ligne;
61         }
62     }
63 }
64
65
66 int TDC_estPresent(TableDeCodage tab, char car) {
67     TableDeCodage temp;
68     if (TDC_estVide(tab)){
69         return 0;
70     }
71     if ( tab.ligne->c==car) {
72         return 1;
73     }
74     else {
75         temp.ligne = tab.ligne->listeSuivante;
76         return TDC_estPresent(temp, car);
77     }
78 }
79
80 bool TDC_estVide(TableDeCodage tab){
81     return tab.ligne == NULL;
82 }
83
84
85 char* TDC_afficherTdc(TableDeCodage* tdc){
86     Ligne *actuel = tdc->ligne;
87     printf("la table de codage est: \n");
88     while (actuel != NULL) {
89         printf(" | Caracter is: %c  ,CodeBinaire is: %
              s |\n", actuel->c,CB_codeBinaireToStr(&
              actuel->code));
```

```
90          actuel = actuel->listeSuivante;
91      }
92      return NULL;
93  }
```

11.1.7 Compression

Listing 7: Compression.c

```
1   #include "Octet.h"
2   #include "FileDePriorite.h"
3   #include <stdio.h>
4   #include <stdlib.h>
5   #include "CodeBinaire.h"
6   #include "Statistiques.h"
7   #include "ArbreHuffman.h"
8   #include "FileDePriorite.h"
9   #include "TableDeCodage.h"
10
11
12
13
14  FILE* compresser(const char* fileName){
15      FILE* file = fopen(fileName,"r");
16      if (file == NULL) {
17          perror("Erreur lors de l'ouverture du fichier"
                );
18          exit(EXIT_FAILURE);
19      }
20      FileDePriorite fp= FDP_fileDePriorite();
21
22      const char *filename = "compressedFile.txt";
23      FILE *compressedFile = fopen(filename, "w");
24      if (compressedFile == NULL) {
25          fprintf(stderr, "Impossible d'ouvrir le
                fichier %s en écriture.\n", filename);
26      }
27
28      int occurrences[256] = {0};
29      int caracteresSansRepetition[256] = {0};
30      int nbCaracteresSansRepetition = 0;
31      int nbCaracAvecRepetition=0;
32
33      int caractere;
34      while ((caractere = fgetc(file)) != EOF) {
```

```
35          nbCaracAvecRepetition++;
36          occurrences[caractere]++;
37          if (occurrences[caractere] == 1) {
38              caracteresSansRepetition[
                    nbCaracteresSansRepetition++] =
                    caractere;
39          }
40      }
41      fprintf(compressedFile,"%d\n",
            nbCaracAvecRepetition-1);
42
43      for (int i = 0; i < nbCaracteresSansRepetition; i
            ++) {
44          Statistiques stat = S_statistique(
                caracteresSansRepetition[i],occurrences[
                caracteresSansRepetition[i]]);
45          if(i==nbCaracteresSansRepetition-1){
46              fprintf(compressedFile,"%c%d\n",
                    S_obtenirCaractere(stat),
                    S_obtenirPonderation(stat));
47          }
48          else{
49              fprintf(compressedFile,"%c%d",
                    S_obtenirCaractere(stat),
                    S_obtenirPonderation(stat));
50          }
51          ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
52          FDP_enfiler(&fp,arbre);
53      }
54
55
56
57      while(fp.longueur != 1){
58          ArbreHuffman arbre1 = FDP_obtenirElement(fp,
                fp.longueur);
59          ArbreHuffman arbre2 = FDP_obtenirElement(fp,
                fp.longueur - 1);
60          FDP_defiler(&fp);
61          FDP_defiler(&fp);
62          ArbreHuffman* arbreFusionne1 = AH_fusionner(&
                arbre1,&arbre2);
63          ArbreHuffman* copy = copierArbreHuffman(
                arbreFusionne1);
64          FDP_enfiler(&fp, copy);
65      }
66
```

```
67
68
69
70      ArbreHuffman ArbreFinal = FDP_obtenirElement(fp,0)
            ;
71      TableDeCodage* tableDeCodageFinale =
            TDC_obtenirTableDeCodage();
72      for (int i=0 ; i<nbCaracteresSansRepetition;i++){
73          char c = caracteresSansRepetition[i];
74          CodeBinaire cb = CB_CbFromAH(&ArbreFinal,c);
75          TDC_ajouter(tableDeCodageFinale,c,&cb);
76      }
77
78      TDC_afficherTdc(tableDeCodageFinale);
79
80
81
82      fseek(file, 0, SEEK_SET);
83      Octet firstOctet = O_creerOctet();
84      char car;
85      while ((car = fgetc(file)) != EOF) {
86          if(!O_currentisFull()){
87              CodeBinaire* code = TDC_obtenirCode(*
                    tableDeCodageFinale,car);
88              firstOctet = O_encoder(firstOctet,*code);
89
90          }
91          else{
92              //printf("octet is is %s\n", O_octetToStr
                    (&firstOctet));
93              fprintf(compressedFile,"%s.",O_bitsToStr(&
                    firstOctet));
94              firstOctet = O_creerOctet();
95              //printf("J'ai creer un nv octet\n");
96              CodeBinaire* code = TDC_obtenirCode(*
                    tableDeCodageFinale,car);
97              firstOctet =O_encoder(firstOctet,*code);
98          }
99      }
100     //printf("octet is is %s\n", O_octetToStr(&
            firstOctet));
101     fprintf(compressedFile,"%s.",O_bitsToStr(&
            firstOctet));
102     if(O_obtenirMostImportantBitRestant()<= 7){
103         Octet octet = O_inverser(O_obtenirBitRestants
                ());
```

```
104             fprintf(compressedFile,"%s",O_bitsToStr(&octet
                    ));
105         }
106
107         fclose(file);
108
109         return compressedFile;
110
111 }
112
113 void null(){
114     printf("aa");
115 }
```

11.1.8 Decompression

Listing 8: Decompression.c

```
1  #include "Octet.h"
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "CodeBinaire.h"
5  #include "Statistiques.h"
6  #include "ArbreHuffman.h"
7  #include "FileDePriorite.h"
8  #include "TableDeCodage.h"
9  #define TAILLE_MAX 1000
10
11
12 FileDePriorite DECOMP_fileFromFILE(FileDePriorite fp,
       char caracter,int ponderation){
13     Statistiques stat= S_statistique(caracter,
           ponderation);
14     ArbreHuffman* arbre= AH_arbreDeHuffman(stat);
15     FDP_enfiler(&fp,arbre);
16     return fp;
17 }
18
19
20 ArbreHuffman DECOMP_arbrePrincipale(FileDePriorite fp)
       {
21
22     while(fp.longueur != 1){
23         ArbreHuffman arbre1 = FDP_obtenirElement(fp,
               fp.longueur);
```

```
24          ArbreHuffman arbre2 = FDP_obtenirElement(fp,
                fp.longueur - 1);
25          FDP_defiler(&fp);
26          FDP_defiler(&fp);
27          ArbreHuffman* arbreFusionne1 = AH_fusionner(&
                arbre1,&arbre2);
28          ArbreHuffman* copy = copierArbreHuffman(
                arbreFusionne1);
29          FDP_enfiler(&fp, copy);
30      }
31
32      return FDP_obtenirElement(fp,0);
33  }
34
35
36  FILE* DECOMP_decompresser(const char* fileName){
37      FILE *file = fopen(fileName, "r");
38      char tailleStr[TAILLE_MAX] = "";
39      int taille = 1;
40
41      if (file == NULL) {
42          perror("Erreur lors de l'ouverture du fichier"
                );
43          exit(EXIT_FAILURE);
44      }
45
46
47      if(fgets(tailleStr, TAILLE_MAX, file) != NULL){
48          taille= atoi(tailleStr);
49      }
50
51      char c;
52      FileDePriorite fp= FDP_fileDePriorite();
53      while((c = fgetc(file)) != '\n'){
54          char statCaracter = c;
55          c= fgetc(file);
56          int statPonderation = atoi(&c);
57          fp = DECOMP_fileFromFILE(fp,statCaracter,
                statPonderation);
58      }
59
60      const char *decompressed = "decompressedFile.txt";
61      FILE* decompresssedFile = fopen(decompressed,"w");
62      ArbreHuffman arbrePrincipale =
            DECOMP_arbrePrincipale(fp);
63      ArbreHuffman *tmp = &arbrePrincipale;
```

```
64      char car;
65      while((car = fgetc(file)) != EOF){
66          if(car== '.'){
67              continue;
68          }
69
70          else{
71              if(car== '0') tmp = tmp->filsGauche;
72              else if (car =='1') tmp=tmp->filsDroit;
73          }
74          if (AH_estFeuille(*tmp)){
75              fprintf(decompresssedFile,"%c",
                    AH_obtenirCaracter(*tmp));
76              tmp = &arbrePrincipale;
77
78          }
79      }
80      fclose(file);
81      return NULL;
82  }
```

11.2 Tests unitaires

11.2.1 Arbre Huffman

Listing 9: testArbreHuffman.c

```
1   #include <stdio.h>
2   #include <stdbool.h>
3   #include <CUnit/Basic.h>
4   #include "ArbreHuffman.h"
5   #include "Statistiques.h"
6
7   void test_creation_arbreHuffman(){
8       Statistiques stat = S_statistique('A',5);
9       ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
10      CU_ASSERT_EQUAL(AH_obtenirPonderation(*arbre),5);
11      CU_ASSERT_PTR_NULL(AH_obtenirFilsDroit(arbre));
12      CU_ASSERT_PTR_NULL(AH_obtenirFilsGauche(arbre));
13  }
14
15  void test_obtenirOccurences() {
16      Statistiques stat = S_statistique('A',5);
17      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
18      int occurences = AH_obtenirPonderation(*arbre);
```

```
19      CU_ASSERT_EQUAL(occurences, S_obtenirPonderation(
            stat));
20  }
21
22  void test_fusionner(){
23      Statistiques stat = S_statistique('A',5);
24      ArbreHuffman* arbre1 = AH_arbreDeHuffman(stat);
25      Statistiques stat2 = S_statistique('E',2);
26      ArbreHuffman* arbre2 = AH_arbreDeHuffman(stat2);
27      ArbreHuffman* arbreF = AH_fusionner(arbre1,arbre2)
            ;
28      CU_ASSERT_EQUAL(AH_obtenirPonderation(*arbreF),
            S_obtenirPonderation(arbre1->statistiques)+
            S_obtenirPonderation(arbre2->statistiques));
29      CU_ASSERT_EQUAL(AH_obtenirFilsDroit(arbreF),arbre1
            );
30      CU_ASSERT_EQUAL(AH_obtenirFilsGauche(arbreF),
            arbre2);
31  }
32
33  void test_fixerFilsGauche_et_obtenirFilsGauche() {
34      Statistiques stat = S_statistique('A',5);
35      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
36      Statistiques statFils = S_statistique('T',6);
37      ArbreHuffman* filsGauche = AH_arbreDeHuffman(
            statFils);
38      AH_fixerFilsGauche(arbre, filsGauche);
39      ArbreHuffman* obtenu = AH_obtenirFilsGauche(arbre)
            ;
40      CU_ASSERT_PTR_EQUAL(obtenu, filsGauche);
41  }
42
43  void test_fixerFilsDroit_et_obtenirFilsDroit() {
44      Statistiques stat = S_statistique('A',5);
45      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
46      Statistiques statFils = S_statistique('T',6);
47      ArbreHuffman* filsDroit = AH_arbreDeHuffman(
            statFils);
48      AH_fixerFilsDroit(arbre, filsDroit);
49      ArbreHuffman* obtenu = AH_obtenirFilsDroit(arbre);
50      CU_ASSERT_PTR_EQUAL(obtenu, filsDroit);
51  }
52
53  void test_estVide() {
54      Statistiques stat;
55      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
```

```
56      CU_ASSERT_PTR_NULL(AH_obtenirFilsGauche(arbre));
57  }
58
59  void test_estFeuille() {
60      Statistiques stat = S_statistique('A',5);
61      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
62      CU_ASSERT_TRUE(AH_estFeuille(*arbre));
63  }
64
65  void test_estPresent_caractere_present_a_la_racine() {
66      Statistiques stat = S_statistique('A',5);
67      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
68      char caractere = 'A';
69
70      CU_ASSERT_TRUE(AH_estPresent(arbre, caractere));
71  }
72
73  void
        test_estPresent_caractere_present_sous_arbre_gauche
        () {
74      Statistiques stat = S_statistique('C',7);
75      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
76      Statistiques statFils = S_statistique('A',5);
77      ArbreHuffman* arbreFils = AH_arbreDeHuffman(
            statFils);
78      AH_fixerFilsGauche(arbre,arbreFils);
79      CU_ASSERT_TRUE(AH_estPresent(arbre, 'A'));
80  }
81
82  void
        test_estPresent_caractere_present_sous_arbre_droit
        () {
83      Statistiques stat = S_statistique('C',7);
84      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
85      Statistiques statFils = S_statistique('A',5);
86      ArbreHuffman* arbreFils = AH_arbreDeHuffman(
            statFils);
87      char caractere = 'A';
88      AH_fixerFilsDroit(arbre,arbreFils);
89      CU_ASSERT_TRUE(AH_estPresent(arbre, caractere));
90  }
91
92  void test_estPresent_caractere_absent() {
93      Statistiques stat = S_statistique('A',5);
94      ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
95      char caractere = 'B';
```

```
96
97      CU_ASSERT_FALSE(AH_estPresent(arbre, caractere));
98  }
99
100 int main() {
101     CU_pSuite pSuite = NULL;
102
103     if (CUE_SUCCESS != CU_initialize_registry())
104         return CU_get_error();
105
106     pSuite = CU_add_suite("Tests ArbreHuffman", NULL,
            NULL);
107     if (NULL == pSuite) {
108         CU_cleanup_registry();
109         return CU_get_error();
110     }
111
112     if ((NULL == CU_add_test(pSuite, "Test création
            ArbreHuffman", test_creation_arbreHuffman))
113         || (NULL == CU_add_test(pSuite, "Test
                obtenirOccurences", test_obtenirOccurences)
                )
114         || (NULL == CU_add_test(pSuite, "Test
                fusionner", test_fusionner))
115         || (NULL == CU_add_test(pSuite, "Test
                fixerFilsGauche et obtenirFilsGauche",
                test_fixerFilsGauche_et_obtenirFilsGauche))
116         || (NULL == CU_add_test(pSuite, "Test
                fixerFilsDroit et obtenirFilsDroit",
                test_fixerFilsDroit_et_obtenirFilsDroit))
117         || (NULL == CU_add_test(pSuite, "Test estVide"
                , test_estVide))
118         || (NULL == CU_add_test(pSuite, "Test
                estFeuille", test_estFeuille))
119         || (NULL == CU_add_test(pSuite, "Test caractè
                re présent á la racine",
                test_estPresent_caractere_present_a_la_racine
                ))
120         || (NULL == CU_add_test(pSuite, "Test caractè
                re présent sous arbre gauche",
                test_estPresent_caractere_present_sous_arbre_gauche
                ))
121         || (NULL == CU_add_test(pSuite, "Test caractè
                re présent sous arbre droit",
                test_estPresent_caractere_present_sous_arbre_droit
                ))
```

```
122         || (NULL == CU_add_test(pSuite, "Test caractè
                re absent",
                test_estPresent_caractere_absent))) {
123         CU_cleanup_registry();
124         return CU_get_error();
125     }
126
127     CU_basic_set_mode(CU_BRM_VERBOSE);
128     CU_basic_run_tests();
129     printf("\n");
130     CU_basic_show_failures(CU_get_failure_list());
131     printf("\n\n");
132
133     CU_cleanup_registry();
134     return CU_get_error();
135 }
```

11.2.2 Code Binaire

Listing 10: testCodeBinaire.c

```
1  #include <CUnit/CUnit.h>
2  #include <CUnit/Basic.h>
3  #include "CodeBinaire.h"
4  #include <stdlib.h>
5
6  // Fonctions de configuration et de nettoyage pour
       CUnit
7  int init_suite(void) {
8      return 0;
9  }
10
11 int clean_suite(void) {
12     return 0;
13 }
14
15 // Tests pour CB_estVide
16 void test_CB_estVide(void) {
17     CodeBinaire code1 = CB_creerCodeBinaire(4);
18     CU_ASSERT_FALSE(CB_estVide(code1));
19
20     CodeBinaire code2 = CB_creerCodeBinaire(8);
21     CU_ASSERT_FALSE(CB_estVide(code2));
22
23     CB_libererCodeBinaire(&code1);
```

```
24      CB_libererCodeBinaire(&code2);
25  }
26
27  // Tests pour CB_creerCodeBinaire
28  void test_CB_creerCodeBinaire(void) {
29      CodeBinaire code = CB_creerCodeBinaire(4);
30      CU_ASSERT_EQUAL(code.taille, 4);
31      for (size_t i = 0; i < code.taille; ++i) {
32          CU_ASSERT_EQUAL(code.bits[i], BITA0);
33      }
34      CB_libererCodeBinaire(&code);
35  }
36
37  // Tests pour CB_remplirCodeBinaire
38  void test_CB_remplirCodeBinaire(void) {
39      CodeBinaire code;
40      code.taille = 4;
41      code.bits = malloc(4 * sizeof(Bit));
42      const char * bits =  "1101";
43
44      CB_remplirCodeBinaire(&code,bits);
45
46      for (int i = 0; i < code.taille; ++i) {
47          if(strcmp(&bits[i],"0"),0) CU_ASSERT_EQUAL(
                code.bits[i],BITA0);
48          if(strcmp(&bits[i],"1"),0) CU_ASSERT_EQUAL(
                code.bits[i],BITA1);
49      }
50
51      CB_libererCodeBinaire(&code);
52  }
53
54  // Tests pour CB_obtenirIemeBit
55  void test_CB_obtenirIemeBit(void) {
56      CodeBinaire code = CB_creerCodeBinaire(4);
57
58      for (size_t i = 0; i < code.taille; ++i) {
59          CU_ASSERT_EQUAL(CB_obtenirIemeBit(code, i),
                BITA0);
60      }
61
62      CB_libererCodeBinaire(&code);
63  }
64
65  // Tests pour CB_concatener
66  void test_CB_concatener(void) {
```

```
67      CodeBinaire code1 = CB_creerCodeBinaire(2);
68      CodeBinaire code2 = CB_creerCodeBinaire(3);
69
70      CodeBinaire resultat = CB_concatener(&code1, &
            code2);
71
72      CU_ASSERT_EQUAL(resultat.taille, code1.taille +
            code2.taille);
73
74      for (size_t i = 0; i < code1.taille; ++i) {
75          CU_ASSERT_EQUAL(resultat.bits[i], BITA0);
76      }
77
78      for (size_t i = 0; i < code2.taille; ++i) {
79          CU_ASSERT_EQUAL(resultat.bits[code1.taille + i
                ], BITA0);
80      }
81
82      CB_libererCodeBinaire(&code1);
83      CB_libererCodeBinaire(&code2);
84      CB_libererCodeBinaire(&resultat);
85  }
86
87  // Tests pour CB_codeBinaireToStr
88  void test_CB_codeBinaireToStr(void) {
89      CodeBinaire code = CB_creerCodeBinaire(3);
90      CU_ASSERT_STRING_EQUAL(CB_codeBinaireToStr(&code),
            "000");
91
92      CB_remplirCodeBinaire(&code, "101");
93      CU_ASSERT_STRING_EQUAL(CB_codeBinaireToStr(&code),
            "101");
94
95      CB_libererCodeBinaire(&code);
96  }
97
98  void test_CB_CbFromAH(){
99      Statistiques stat = S_statistique('X', 5);
100     ArbreHuffman* arbre = AH_arbreDeHuffman(stat);
101
102     Statistiques statA = S_statistique('A', 3);
103     ArbreHuffman* arbreA = AH_arbreDeHuffman(statA);
104
105     Statistiques statB = S_statistique('B', 3);
106     ArbreHuffman* arbreB = AH_arbreDeHuffman(statB);
107
```

```
108     Statistiques statC = S_statistique('C', 3);
109     ArbreHuffman* arbreC = AH_arbreDeHuffman(statC);
110 
111     Statistiques statD = S_statistique('D', 4);
112     ArbreHuffman* arbreD = AH_arbreDeHuffman(statD);
113 
114     Statistiques statE = S_statistique('E', 5);
115     ArbreHuffman* arbreE = AH_arbreDeHuffman(statE);
116 
117     Statistiques statF = S_statistique('F', 3);
118     ArbreHuffman* arbreF = AH_arbreDeHuffman(statF);
119 
120     AH_fixerFilsGauche(arbre, arbreA);
121     AH_fixerFilsDroit(arbre, arbreB);
122     AH_fixerFilsGauche(arbreA , arbreC);
123     AH_fixerFilsDroit(arbreA, arbreD);
124     AH_fixerFilsGauche(arbreB , arbreE);
125     AH_fixerFilsDroit(arbreB, arbreF);
126 
127     CodeBinaire cb = CB_CbFromAH(arbre,'E');
128     CU_ASSERT_STRING_EQUAL(CB_codeBinaireToStr(&cb), "
        10");
129 
130 }
131 
132 int main() {
133     // Initialisation de CUnit
134     if (CUE_SUCCESS != CU_initialize_registry()) {
135         return CU_get_error();
136     }
137 
138     // Ajout des suites de tests
139     CU_pSuite suite_CB = CU_add_suite("Suite
        CodeBinaire", init_suite, clean_suite);
140     CU_add_test(suite_CB, "Test CB_estVide",
        test_CB_estVide);
141     CU_add_test(suite_CB, "Test CB_creerCodeBinaire",
        test_CB_creerCodeBinaire);
142     CU_add_test(suite_CB, "Test CB_remplirCodeBinaire"
        , test_CB_remplirCodeBinaire);
143     CU_add_test(suite_CB, "Test CB_obtenirIemeBit",
        test_CB_obtenirIemeBit);
144     CU_add_test(suite_CB, "Test CB_concatener",
        test_CB_concatener);
145     CU_add_test(suite_CB, "Test CB_codeBinaireToStr",
        test_CB_codeBinaireToStr);
```

```
146     CU_add_test(suite_CB, "Test CB_CbFromAH",
            test_CB_CbFromAH);
147
148     // Exécution des tests
149     CU_basic_run_tests();
150
151     // Nettoyage de CUnit
152     CU_cleanup_registry();
153
154     return CU_get_error();
155 }
```

11.2.3 Statistiques

Listing 11: testFileDePriorite.c

```
1  #include<stdio.h>
2  #include <stdlib.h>
3  #include<stdbool.h>
4  #include <CUnit/Basic.h>
5  #include "FileDePriorite.h"
6  #include "ArbreHuffman.h"
7  #include "Statistiques.h"
8
9  void test_creation_file() {
10     FileDePriorite file;
11     file = FDP_fileDePriorite();
12     CU_ASSERT_PTR_NULL(file.debut);
13     CU_ASSERT_EQUAL(file.longueur,0);
14 }
15
16 void testEnfiler_milieu() {
17     FileDePriorite file1;
18     FileDePriorite file2;
19     file1 = FDP_fileDePriorite();
20     file2 = FDP_fileDePriorite();
21     Statistiques stat1 = S_statistique('a', 3);
22     Statistiques stat2 = S_statistique('b', 4);
23     Statistiques stat3 = S_statistique('c', 7);
24     FDP_enfiler(&file1,AH_arbreDeHuffman(stat1));
25     FDP_enfiler(&file1,AH_arbreDeHuffman(stat2));
26     FDP_enfiler(&file1,AH_arbreDeHuffman(stat3));
27
28     FDP_enfiler(&file2,AH_arbreDeHuffman(stat1));
29     FDP_enfiler(&file2,AH_arbreDeHuffman(stat3));
```

```
30
31      FDP_enfiler(&file2,AH_arbreDeHuffman(stat2));
32
33      for (int i = 1; i <= file1.longueur; i++) {
34          ArbreHuffman arbre1 = FDP_obtenirElement(file1
                , i);
35          ArbreHuffman arbre2 = FDP_obtenirElement(file2
                , i);
36          CU_ASSERT_EQUAL(AH_obtenirPonderation(arbre1),
                 AH_obtenirPonderation(arbre2));
37          CU_ASSERT_EQUAL(AH_obtenirCaracter(arbre1),
                AH_obtenirCaracter(arbre2));
38      }
39      CU_ASSERT_EQUAL(file1.longueur,file2.longueur);
40  }
41
42  void testEnfiler_debut() {
43      FileDePriorite file1;
44      FileDePriorite file2;
45      file1 = FDP_fileDePriorite();
46      file2 = FDP_fileDePriorite();
47      Statistiques stat1 = S_statistique('a', 3);
48      Statistiques stat2 = S_statistique('b', 4);
49      Statistiques stat3 = S_statistique('c', 7);
50      FDP_enfiler(&file1,AH_arbreDeHuffman(stat1));
51      FDP_enfiler(&file1,AH_arbreDeHuffman(stat2));
52      FDP_enfiler(&file1,AH_arbreDeHuffman(stat3));
53
54      FDP_enfiler(&file2,AH_arbreDeHuffman(stat2));
55      FDP_enfiler(&file2,AH_arbreDeHuffman(stat1));
56
57      FDP_enfiler(&file2,AH_arbreDeHuffman(stat3));
58
59      for (int i = 1; i <= file1.longueur; i++) {
60          ArbreHuffman arbre1 = FDP_obtenirElement(file1
                , i);
61          ArbreHuffman arbre2 = FDP_obtenirElement(file2
                , i);
62          CU_ASSERT_EQUAL(AH_obtenirPonderation(arbre1),
                 AH_obtenirPonderation(arbre2));
63          CU_ASSERT_EQUAL(AH_obtenirCaracter(arbre1),
                AH_obtenirCaracter(arbre2));
64      }
65      CU_ASSERT_EQUAL(file1.longueur,file2.longueur);
66  }
67
```

```
68 void testEnfiler_fin() {
69    FileDePriorite file1;
70     FileDePriorite file2;
71     file1 = FDP_fileDePriorite();
72     file2 = FDP_fileDePriorite();
73     Statistiques stat1 = S_statistique('a', 3);
74     Statistiques stat2 = S_statistique('b', 4);
75     Statistiques stat3 = S_statistique('c', 7);
76     FDP_enfiler(&file1,AH_arbreDeHuffman(stat1));
77     FDP_enfiler(&file1,AH_arbreDeHuffman(stat2));
78     FDP_enfiler(&file1,AH_arbreDeHuffman(stat3));
79 
80     FDP_enfiler(&file2,AH_arbreDeHuffman(stat3));
81     FDP_enfiler(&file2,AH_arbreDeHuffman(stat2));
82 
83     FDP_enfiler(&file2,AH_arbreDeHuffman(stat1));
84 
85     for (int i = 1; i <= file1.longueur; i++) {
86         ArbreHuffman arbre1 = FDP_obtenirElement(file1
               , i);
87         ArbreHuffman arbre2 = FDP_obtenirElement(file2
               , i);
88         CU_ASSERT_EQUAL(AH_obtenirPonderation(arbre1),
                AH_obtenirPonderation(arbre2));
89         CU_ASSERT_EQUAL(AH_obtenirCaracter(arbre1),
               AH_obtenirCaracter(arbre2));
90     }
91     CU_ASSERT_EQUAL(file1.longueur,file2.longueur);
92 }
93 
94 void testDefiler() {
95     FileDePriorite file1;
96     FileDePriorite file2;
97     file1 = FDP_fileDePriorite();
98     file2 = FDP_fileDePriorite();
99     Statistiques stat1 = S_statistique('a', 3);
100    Statistiques stat2 = S_statistique('b', 4);
101    Statistiques stat3 = S_statistique('c', 7);
102    FDP_enfiler(&file1,AH_arbreDeHuffman(stat2));
103    FDP_enfiler(&file1,AH_arbreDeHuffman(stat3));
104 
105    FDP_enfiler(&file2,AH_arbreDeHuffman(stat1));
106    FDP_enfiler(&file2,AH_arbreDeHuffman(stat2));
107    FDP_enfiler(&file2,AH_arbreDeHuffman(stat3));
108 
109    FDP_defiler(&file2);
```

```
110
111     for (int i = 1; i <= file1.longueur; i++) {
112         ArbreHuffman arbre1 = FDP_obtenirElement(file1
                , i);
113         ArbreHuffman arbre2 = FDP_obtenirElement(file2
                , i);
114         CU_ASSERT_EQUAL(AH_obtenirPonderation(arbre1),
                 AH_obtenirPonderation(arbre2));
115         CU_ASSERT_EQUAL(AH_obtenirCaracter(arbre1),
                AH_obtenirCaracter(arbre2));
116     }
117     CU_ASSERT_EQUAL(file1.longueur,file2.longueur);
118 }
119
120 void test_obtenirIElement_debut() {
121     FileDePriorite file;
122     file = FDP_fileDePriorite();
123     Statistiques stat1 = S_statistique('a', 3);
124     Statistiques stat2 = S_statistique('b', 4);
125     Statistiques stat3 = S_statistique('c', 7);
126
127     FDP_enfiler(&file,AH_arbreDeHuffman(stat1));
128     FDP_enfiler(&file,AH_arbreDeHuffman(stat2));
129     FDP_enfiler(&file,AH_arbreDeHuffman(stat3));
130
131     ArbreHuffman arbre_obtenu = FDP_obtenirElement(
            file,1);
132     ArbreHuffman arbre_theorique = *AH_arbreDeHuffman(
            stat3);
133
134     CU_ASSERT_PTR_EQUAL(arbre_obtenu.filsDroit,
            arbre_theorique.filsDroit);
135     CU_ASSERT_PTR_EQUAL(arbre_obtenu.filsGauche,
            arbre_theorique.filsGauche);
136     CU_ASSERT_EQUAL(arbre_obtenu.statistiques.caracter
            ,arbre_theorique.statistiques.caracter);
137     CU_ASSERT_EQUAL(arbre_obtenu.statistiques.
            ponderation,arbre_theorique.statistiques.
            ponderation);
138 }
139
140 void test_obtenirIElement_milieu() {
141     FileDePriorite file;
142     file = FDP_fileDePriorite();
143     Statistiques stat1 = S_statistique('a', 3);
144     Statistiques stat2 = S_statistique('b', 4);
```

```
145     Statistiques stat3 = S_statistique('c', 7);
146
147     FDP_enfiler(&file,AH_arbreDeHuffman(stat1));
148     FDP_enfiler(&file,AH_arbreDeHuffman(stat2));
149     FDP_enfiler(&file,AH_arbreDeHuffman(stat3));
150
151     ArbreHuffman arbre_obtenu = FDP_obtenirElement(
            file,2);
152     ArbreHuffman arbre_theorique = *AH_arbreDeHuffman(
            stat2);
153
154     CU_ASSERT_PTR_EQUAL(arbre_obtenu.filsDroit,
            arbre_theorique.filsDroit);
155     CU_ASSERT_PTR_EQUAL(arbre_obtenu.filsGauche,
            arbre_theorique.filsGauche);
156     CU_ASSERT_EQUAL(arbre_obtenu.statistiques.caracter
            ,arbre_theorique.statistiques.caracter);
157     CU_ASSERT_EQUAL(arbre_obtenu.statistiques.
            ponderation,arbre_theorique.statistiques.
            ponderation);
158 }
159
160 void test_obtenirIElement_fin() {
161     FileDePriorite file;
162     file = FDP_fileDePriorite();
163     Statistiques stat1 = S_statistique('a', 3);
164     Statistiques stat2 = S_statistique('b', 4);
165     Statistiques stat3 = S_statistique('c', 7);
166
167     FDP_enfiler(&file,AH_arbreDeHuffman(stat1));
168     FDP_enfiler(&file,AH_arbreDeHuffman(stat2));
169     FDP_enfiler(&file,AH_arbreDeHuffman(stat3));
170
171     ArbreHuffman arbre_obtenu = FDP_obtenirElement(
            file,3);
172     ArbreHuffman arbre_theorique = *AH_arbreDeHuffman(
            stat1);
173
174     CU_ASSERT_PTR_EQUAL(arbre_obtenu.filsDroit,
            arbre_theorique.filsDroit);
175     CU_ASSERT_PTR_EQUAL(arbre_obtenu.filsGauche,
            arbre_theorique.filsGauche);
176     CU_ASSERT_EQUAL(arbre_obtenu.statistiques.caracter
            ,arbre_theorique.statistiques.caracter);
177     CU_ASSERT_EQUAL(arbre_obtenu.statistiques.
            ponderation,arbre_theorique.statistiques.
```

```
        ponderation);
}

void test_nouvellefile() {
    FileDePriorite file;
    FileDePriorite file_f;
    Statistiques stat1 = S_statistique('a', 3);
    Statistiques stat2 = S_statistique('b', 4);
    Statistiques stat3 = S_statistique('c', 7);
    Statistiques stat4 = S_statistique(' ',7);
    ArbreHuffman arbre1 = *AH_arbreDeHuffman(stat1);
    ArbreHuffman arbre2 = *AH_arbreDeHuffman(stat2);
    ArbreHuffman arbre3 = *AH_arbreDeHuffman(stat3);
    ArbreHuffman arbre_f = *AH_arbreDeHuffman(stat4);
    AH_fixerFilsGauche(&arbre_f, &arbre1);
    AH_fixerFilsDroit(&arbre_f, &arbre2);

    file = FDP_fileDePriorite();
    FDP_enfiler(&file,&arbre1);
    FDP_enfiler(&file,&arbre2);
    FDP_enfiler(&file,&arbre3);

    file_f = FDP_fileDePriorite();
    FDP_enfiler(&file_f,&arbre3);
    FDP_enfiler(&file_f,&arbre_f);

    FDP_nouvelleFile(&file);

    for (int i = 1; i <= file_f.longueur; i++) {
        ArbreHuffman arbre_file = FDP_obtenirElement(
            file, i);
        ArbreHuffman arbre_file_f = FDP_obtenirElement
            (file_f, i);
        CU_ASSERT_EQUAL(AH_obtenirPonderation(
            arbre_file), AH_obtenirPonderation(
            arbre_file_f));
        CU_ASSERT_EQUAL(AH_obtenirCaracter(arbre_file)
            , AH_obtenirCaracter(arbre_file_f));
    }
    CU_ASSERT_EQUAL(file.longueur, file_f.longueur);
}


int main (int argc, char** argv) {
    CU_pSuite pSuite = NULL;

```

```
218     if (CUE_SUCCESS != CU_initialize_registry())
219         return CU_get_error();
220 
221     pSuite = CU_add_suite("Tests FileDePriorite", NULL
        , NULL);
222     if (NULL == pSuite) {
223         CU_cleanup_registry();
224         return CU_get_error();
225     }
226 
227     if ((NULL == CU_add_test(pSuite, "Test création
        File", test_creation_file)
228         || (NULL == CU_add_test(pSuite, "Test enfiler
            milieu", testEnfiler_milieu))
229         || (NULL == CU_add_test(pSuite, "Test enfiler
            début", testEnfiler_debut))
230         || (NULL == CU_add_test(pSuite, "Test enfiler
            fin", testEnfiler_fin))
231         || (NULL == CU_add_test(pSuite, "Test défiler"
            , testDefiler))
232         || (NULL == CU_add_test(pSuite, "Test
            obtenirElement début",
            test_obtenirIElement_debut))
233         || (NULL == CU_add_test(pSuite, "Test
            obtenirElement milieu",
            test_obtenirIElement_milieu))
234         || (NULL == CU_add_test(pSuite, "Test
            obtenirElement fin",
            test_obtenirIElement_fin))
235         || (NULL == CU_add_test(pSuite, "Test
            nouvelleFile", test_nouvellefile)))) {
236         CU_cleanup_registry();
237         return CU_get_error();
238     }
239 
240     CU_basic_set_mode(CU_BRM_VERBOSE);
241     CU_basic_run_tests();
242     printf("\n");
243     CU_basic_show_failures(CU_get_failure_list());
244     printf("\n\n");
245 
246     CU_cleanup_registry();
247     return CU_get_error();
248 }
```

11.2.4 Octet

Listing 12: testOctet.c

```c
#include<stdio.h>
#include<stdbool.h>
#include <CUnit/Basic.h>
#include <string.h>
#include "Octet.h"
#include "CodeBinaire.h"

void test_creer_Octet() {
    Octet octet = O_creerOctet();
    for (int i=0;i<8;i++){
        CU_ASSERT_EQUAL(O_iemeBit(octet,i), BITA0);
    }
}

void test_fixer_bit(){
    Octet octet = O_creerOctet();
    O_fixerBit(&octet,0,BITA1);
    O_fixerBit(&octet,7,BITA1);
    CU_ASSERT_EQUAL(O_iemeBit(octet,0), BITA1);
    CU_ASSERT_EQUAL(O_iemeBit(octet,7), BITA1);
}

void test_encoder(){
    CodeBinaire code = CB_creerCodeBinaire(1);
    CB_remplirCodeBinaire(&code,"1");
    const char* chaineBinaire = "1010";
    Octet octet = O_strToBits(chaineBinaire);

    for(int i=0; i<7;i++){
        octet = O_encoder(octet,code);
    }

    CU_ASSERT_EQUAL(strcmp(O_bitsToStr(&octet),"
       10101111"), 0);
    Octet octetRes = O_obtenirBitRestants();

    CU_ASSERT_EQUAL(strcmp(O_bitsToStr(&octetRes),"
       00000111"), 0);
}


void test_iemeBit(){
```

```
41      Octet octet = O_creerOctet();
42      O_fixerBit(&octet,7,BITA1);
43      O_fixerBit(&octet, 0,BITA0);
44      CU_ASSERT_EQUAL(O_iemeBit(octet,7), BITA1);
45      CU_ASSERT_EQUAL(O_iemeBit(octet,0), BITA0);
46  }
47
48
49  void test_decaler(){
50      char* octet = "01011100";
51      Octet octetInBits = O_strToBits(octet);
52      //octetInBits= O_decaler(octetInBits);
53      O_decaler(&octetInBits);
54      CU_ASSERT_EQUAL(strcmp(O_bitsToStr(&octetInBits),"
            10111000"), 0);
55  }
56
57  void test_str_to_bits(){
58      char* octetInstr = "11110011";
59      Octet octetInBits= O_strToBits(octetInstr);
60      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,0), BITA1);
61      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,1), BITA1);
62      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,2), BITA1);
63      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,3), BITA1);
64      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,4), BITA0);
65      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,5), BITA0);
66      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,6), BITA1);
67      CU_ASSERT_EQUAL(O_iemeBit(octetInBits,7), BITA1);
68  }
69
70  void test_bits_to_str(){
71      Octet octet = O_creerOctet();
72      O_fixerBit(&octet,0,BITA1);
73      O_fixerBit(&octet,1,BITA1);
74      O_fixerBit(&octet,2,BITA1);
75      O_fixerBit(&octet,3,BITA1);
76      O_fixerBit(&octet,4,BITA1);
77      O_fixerBit(&octet,5,BITA1);
78      O_fixerBit(&octet,6,BITA1);
79      O_fixerBit(&octet,7,BITA1);
80      char* octetInstr= O_bitsToStr(&octet);
81      CU_ASSERT_EQUAL(strcmp(octetInstr,"11111111"), 0);
82
83  }
84
85  int main (int argc, char** argv) {
```

```
86
87      CU_pSuite pSuite = NULL;
88
89      if (CUE_SUCCESS != CU_initialize_registry())
90          return CU_get_error();
91
92      pSuite = CU_add_suite("Tests Octet", NULL, NULL);
93      if (NULL == pSuite) {
94          CU_cleanup_registry();
95          return CU_get_error();
96      }
97
98      if ((NULL == CU_add_test(pSuite, "Test création d'
            octet", test_creer_Octet))
99          || (NULL == CU_add_test(pSuite, "Test fixer
                bit", test_fixer_bit))
100         || (NULL == CU_add_test(pSuite, "Test encoder"
                , test_encoder))
101         || (NULL == CU_add_test(pSuite, "Test obtenir
                ieme bit", test_iemeBit))
102         || (NULL == CU_add_test(pSuite, "Test decaler"
                , test_decaler))
103         || (NULL == CU_add_test(pSuite, "Test string
                to bits", test_str_to_bits))
104         || (NULL == CU_add_test(pSuite, "Test bits to
                string", test_bits_to_str))) {
105         CU_cleanup_registry();
106         return CU_get_error();
107     }
108
109     CU_basic_set_mode(CU_BRM_VERBOSE);
110     CU_basic_run_tests();
111     printf("\n");
112     CU_basic_show_failures(CU_get_failure_list());
113     printf("\n\n");
114
115     CU_cleanup_registry();
116     return CU_get_error();
117
118
119 }
```

11.2.5 FileDePriorite

Listing 13: testStatistiques.c

```c
#include <stdio.h>
#include <CUnit/Basic.h>
#include "Statistiques.h"

// Fonction de test pour vérifier la création d'une statistique
void test_creation_statistique() {
    Statistiques stat = S_statistique('A', 5);
    CU_ASSERT_EQUAL(S_obtenirCaractere(stat), 'A');
    CU_ASSERT_EQUAL(S_obtenirPonderation(stat), 5);
}

int main() {
    CU_pSuite pSuite = NULL;

    if (CUE_SUCCESS != CU_initialize_registry())
        return CU_get_error();

    pSuite = CU_add_suite("Tests Statistiques", NULL, NULL);
    if (NULL == pSuite) {
        CU_cleanup_registry();
        return CU_get_error();
    }

    if (NULL == CU_add_test(pSuite, "Test création Statistique", test_creation_statistique)) {
        CU_cleanup_registry();
        return CU_get_error();
    }

    CU_basic_set_mode(CU_BRM_VERBOSE);
    CU_basic_run_tests();
    printf("\n");
    CU_basic_show_failures(CU_get_failure_list());
    printf("\n\n");

    CU_cleanup_registry();
    return CU_get_error();
}
```

11.2.6 TableDeCodage

Listing 14: testTableDeCodage.c

```c
#include "TableDeCodage.h"
#include <stdio.h>
#include <CUnit/Basic.h>
#include "ArbreHuffman.h"
#include "Statistiques.h"

void test_obtenirTableDeCodage() {
    ArbreHuffman a;
    Statistiques stat;
    CodeBinaire cb=CB_creerCodeBinaire(1);
    CB_remplirCodeBinaire(&cb,"0");
    stat = S_statistique('c',5);
    a = *AH_arbreDeHuffman(stat);
    TableDeCodage* table = TDC_obtenirTableDeCodage(a)
        ;
    char c = 'c';
    CU_ASSERT_STRING_EQUAL(table->ligne->c ,c);
    CU_ASSERT_STRING_EQUAL(CB_codeBinaireToStr(&table
        ->ligne->code),CB_codeBinaireToStr(&cb));
    CU_ASSERT_EQUAL(table->ligne->listeSuivante,NULL);
}

void test_TDC_obtenirCode() {
    TableDeCodage table1, table2, table3;
    CodeBinaire cb=CB_creerCodeBinaire(3);
    table1.ligne->c = 'A';
    CB_remplirCodeBinaire(&cb,"001");
    table1.ligne->code = cb;
    table2.ligne->c = 'B';
    CB_remplirCodeBinaire(&cb,"010");
    table2.ligne->code =  cb;
    table3.ligne->c = 'C';
    CB_remplirCodeBinaire(&cb,"011");
    table3.ligne->code = cb;
    table1.ligne->listeSuivante = table2.ligne;
    table2.ligne->listeSuivante = table3.ligne;

    CodeBinaire* res = TDC_obtenirCode(table1,'B');

    CU_ASSERT_STRING_EQUAL(CB_codeBinaireToStr(&res),
        "010");

}

```

```
42
43 void test_TDC_obtenirCarac() {
44     TableDeCodage table1, table2, table3;
45     CodeBinaire cb=CB_creerCodeBinaire(3);
46     CodeBinaire cbres= CB_creerCodeBinaire(3);
47     table1.ligne->c = 'A';
48     CB_remplirCodeBinaire(&cb,"001");
49     table1.ligne->code = cb;
50     table2.ligne->c = 'B';
51     CB_remplirCodeBinaire(&cb,"010");
52     table2.ligne->code =  cb;
53     table3.ligne->c = 'C';
54     CB_remplirCodeBinaire(&cb,"011");
55     table3.ligne->code = cb;
56     table1.ligne->listeSuivante = table2.ligne;
57     table2.ligne->listeSuivante = table3.ligne;
58     CB_remplirCodeBinaire(&cbres,"010");
59     char res = TDC_obtenirCarac(table1,cbres);
60     CU_ASSERT_EQUAL(res, 'B');
61
62 }
63
64 void test_TDC_ajouter(){
65     TableDeCodage table = {NULL};
66     CodeBinaire cbres= CB_creerCodeBinaire(3);
67     CB_remplirCodeBinaire(&cbres,"001");
68     TDC_ajouter(&table,'A',&cbres);
69     CU_ASSERT_EQUAL(table.ligne->c,'A');
70     CU_ASSERT_STRING_EQUAL(CB_codeBinaireToStr(&table.
          ligne->code),"001")
71     CU_ASSERT_EQUAL(table.ligne->listeSuivante,NULL);
72
73 }
74 // void test_TDC_supprimer(){
75 //     TableDeCodage table1, table2, table3;
76 //     CodeBinaire cb=CB_creerCodeBinaire(3);
77 //     table1.ligne->c = 'A';
78 //     CB_remplirCodeBinaire(&cb,'001');
79 //     table1.ligne->code = cb;
80 //     table2.ligne->c = 'B';
81 //     CB_remplirCodeBinaire(&cb,'010');
82 //     table2.ligne->code =  cb;
83 //     table3.ligne->c = 'C';
84 //     CB_remplirCodeBinaire(&cb,'011');
85 //     table3.ligne->code = cb;
86 //     table1.ligne->listeSuivante = &table2;
```

```
87 //      table2.ligne->listeSuivante = &table3;
88
89 //      CB_remplirCodeBinaire(&cb,'010');
90
91 //      TDC_supprimer(&table1,'B',&cb);
92 //      CB_remplirCodeBinaire(&cb,'011');
93
94 //      CU_ASSERT_PTR_NOT_NULL(&table_test.ligne);
95 //      CU_ASSERT_PTR_NOT_NULL(&table_test.ligne->
       listeSuivante);
96 //      CU_ASSERT_EQUAL(table1.ligne->listeSuivante->c
       ,'C');
97 //      CU_ASSERT_EQUAL(table1.ligne->listeSuivante->
       code,cb);
98 // }
99
100 void test_TDC_estPresent(){
101
102     TableDeCodage table1, table2, table3;
103     CodeBinaire cb=CB_creerCodeBinaire(3);
104     table1.ligne->c = 'A';
105     CB_remplirCodeBinaire(&cb,"001");
106     table1.ligne->code = cb;
107     table2.ligne->c = 'B';
108     CB_remplirCodeBinaire(&cb,"010");
109     table2.ligne->code =  cb;
110     table3.ligne->c = 'C';
111     CB_remplirCodeBinaire(&cb,"011");
112     table3.ligne->code = cb;
113     table1.ligne->listeSuivante = table2.ligne;
114     table2.ligne->listeSuivante = table3.ligne;
115
116     int res_present = TDC_estPresent(table1, 'B');
117     int res_non_present = TDC_estPresent(table1, 'D');
118
119     CU_ASSERT_EQUAL(res_present,1);
120     CU_ASSERT_EQUAL(res_non_present,0);
121 }
122
123 void test_TDC_estVide() {
124
125     TableDeCodage table ={NULL};
126     CodeBinaire cb=CB_creerCodeBinaire(3);
127     CU_ASSERT_TRUE(TDC_estVide(table));
128     table.ligne->c = 'A';
129     CB_remplirCodeBinaire(&cb,"001");
```

```
130     table.ligne->code = cb;
131     CU_ASSERT_FALSE(TDC_estVide(table));
132 
133 }
134 
135 
136 int main() {
137     // Initialiser le framework CUnit
138     CU_initialize_registry();
139 
140     // Ajouter la suite de tests
141     CU_pSuite suite = CU_add_suite("Suite_de_tests",
            NULL, NULL);
142     CU_add_test(suite, "test_obtenirTableDeCodage",
            test_obtenirTableDeCodage);
143     CU_add_test(suite, "test_TDC_obtenirCode",
            test_TDC_obtenirCode);
144     CU_add_test(suite, "test_TDC_obtenirCarac",
            test_TDC_obtenirCarac);
145     CU_add_test(suite, "test_TDC_ajouter",
            test_TDC_ajouter);
146     // CU_add_test(suite, "test_TDC_supprimer",
            test_TDC_supprimer);
147     CU_add_test(suite, "test_TDC_estPresent",
            test_TDC_estPresent);
148     CU_add_test(suite, "test_TDC_estVide",
            test_TDC_estVide);
149 
150     // Exécuter les tests
151     CU_basic_run_tests();
152 
153     // Nettoyer le framework CUnit
154     CU_cleanup_registry();
155 
156     return CU_get_error();
157 }
```

12 Conclusion

Pour conclure ce projet, A FAIRE

12.1 Répartition des tâches

12.1.1 Légende des Croix

- ✔ Thomas : Croix vertes
- ✔ Woody : Croix jaunes
- ✔ Achraf : Croix jaunes
- ✔ Elouen : Croix jaunes

	TAD	CP	CD	Développement	Tests unitaires
ArbreHuffman	X	X	X	X	X
CodeBinaire	X	X	X	X	X
FileDePriorite	X	X	X	X	X
Octet	X	X	X	X	X
Statistiques	X	X	X	X	X
TableDeCodage	X	X	X	X	X
Compression	X	X	X	X	
Decompression	X	X	X	X	